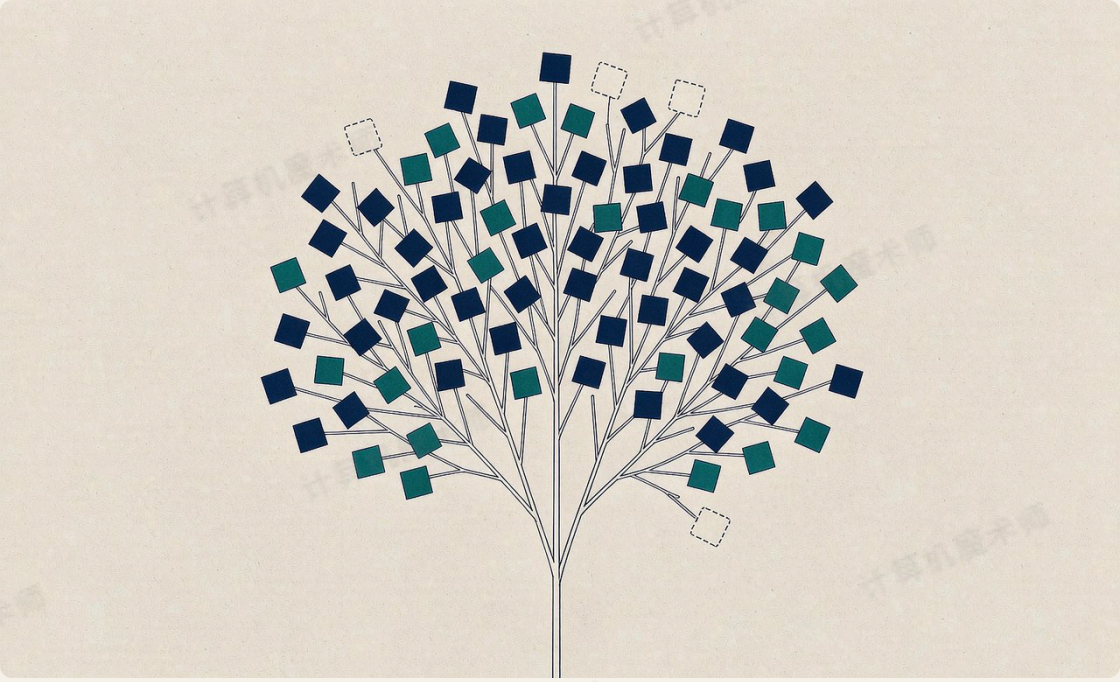
蓝宝书 · 01

DeepSeek Harness



一切皆插件：一个 agent harness 是怎么长出来的

蓝宝书 著

从一个能干活的外壳，讲到一套能搬走的设计

版本 2026-10-09 · v3.1

许可 MIT · 与上游项目一致

目录

序章

§00 序章 最中间的那一层，可以整块抽走

这本书在讲什么

全书地图

怎么读这本书

第 1 部 · 进门

§01 模型明明会写代码，为什么还是干不成活

1.1 小白版：师傅站在门口，进不了厨房

1.2 专业版：外面那一圈要管的六件事

1.3 两种说法的对照

1.4 最容易理解错的地方

1.5 这条能搬去哪

§02 harness 到底是什么，和框架有什么不一样

2.1 小白版：马具、材料店和成品房

2.2 专业版：行业公式、核心不变式与三条分界

2.3 两种说法的对照

2.4 最容易理解错的地方

2.5 这条能搬去哪

§03 同样是外壳，为什么三家造出三种东西

3.1 小白版：三家装修公司接同一个活

3.2 专业版：八维度对比与两种考题

3.3 两种说法的对照

3.4 最容易理解错的地方

3.5 这条能搬去哪

§04 难在哪：能力在变、模型在换、环境不可信、任务还老断

4.1 小白版：队伍开张三个月以后

4.2 专业版：四道难题的性质与排序

4.3 两种说法的对照

4.4 最容易理解错的地方

4.5 这条能搬去哪

第 2 部 · 先看一眼

§05 拆开它，六层各管什么

5.1 小白版：公司的六层楼

5.2 专业版：六层的名字和边界

5.3 两种说法的对照

5.4 最容易理解错的地方

5.5 这条能搬去哪

§06 同一套东西，为什么能装出四种形态

6.1 小白版：规矩叠三层，编制有四套

6.2 专业版：三个角色、一条顺序、四种 preset

6.3 两种说法的对照

6.4 最容易理解错的地方

6.5 这条能搬去哪

第 3 部 · 拆内核

§07 内核里是空的，连 agent 循环都是插件

7.1 小白版：一个只有脑子的师傅

7.2 专业版：插件内核与改动半径

7.3 两种说法的对照

7.4 最容易理解错的地方

7.5 这条能搬去哪

§08 用一样能力，只写它的名字，不用管它在哪

8.1 小白版：报岗位名，不报人名

8.2 专业版：服务键与按名查找

8.3 两种说法的对照

8.4 最容易理解错的地方

8.5 这条能搬去哪

§09 两个插件之间，为什么不直接喊话

9.1 小白版：一块公告栏

9.2 专业版：事件总线与发布订阅

9.3 两种说法的对照

9.4 最容易理解错的地方

9.5 这条能搬去哪

§10 装上去的每一样，都要能干净地拆下来

10.1 小白版：入职发的每一样，离职都要还

10.2 专业版：注册是一件可逆的事

10.3 两种说法的对照
10.4 最容易理解错的地方
10.5 这条能搬去哪

§11 同一件事，为什么只能存一份

11.1 小白版：全公司只有一本总账
11.2 专业版：日志是唯一事实源
11.3 两种说法的对照
11.4 最容易理解错的地方
11.5 这条能搬去哪

第 4 部 · 走一遍

§12 一句问话进来，中间经过了谁

12.1 小白版：师傅修灯的一整天
12.2 专业版：轮次、步骤与一个指挥
12.3 两种说法的对照
12.4 最容易理解错的地方
12.5 这条能搬去哪

§13 一次工具调用，要过哪五道工序

13.1 小白版：一颗螺丝过五道手
13.2 专业版：五段流水线
13.3 两种说法的对照
13.4 最容易理解错的地方
13.5 这条能搬去哪

第 5 部 · 碰边界

§14 它被允许碰什么：三道闸

14.1 小白版：把门的只管三件事
14.2 专业版：收紧、审批、隔离
14.3 两种说法的对照
14.4 最容易理解错的地方
14.5 这条能搬去哪

§15 它出过什么事：四个真实案例

15.1 小白版：把门的出事，四种方式
15.2 专业版：每个案例三步走
15.3 两种说法的对照
15.4 最容易理解错的地方
15.5 这条能搬去哪

第 6 部 · 收

§16 不看代码，怎么知道它真的做到了

16.1 小白版：五个当场能做的试验

16.2 专业版：五条判据与一条链

16.3 两种说法的对照

16.4 最容易理解错的地方

16.5 这条能搬去哪

§17 它在赌什么，代价是什么

17.1 小白版：隔壁行当的一本老账簿

17.2 专业版：时空可组合与两个赌注

17.3 两种说法的对照

17.4 最容易理解错的地方

17.5 这条能搬去哪

附录

附录 A 术语表

一、师傅队伍：全书类比对照

二、构件与角色

三、机制与判断

附录 B 全书图索引

序章

第 1 部 · 进门

第 1 部 · 进门

第 1 部 · 进门

第 1 部 · 进门

第 2 部 · 先看一眼

第 2 部 · 先看一眼

第 3 部 · 拆内核

第 3 部 · 拆内核

第 3 部 · 拆内核

第 3 部 · 拆内核

第 3 部 · 拆内核

第 4 部 · 走一遍

第 4 部 · 走一遍

第 5 部 · 碰边界

第 5 部 · 碰边界

第 6 部 · 收

第 6 部 · 收
附录 C
重印的图
附录 C 概念与代码位置对照
一、概念、代码与键
二、92 个 ctx 键的角色分布
三、想核这些数字时，去哪儿看

序章

§00 序章 最中间的那一层，可以整块抽走

先说一件反常识的事。

这本书讲的那套系统，最中间的那一层可以整块抽走。不是改几行、补几个接口的那种抽法，是整个拿掉，换一副新的放回去，外面所有部分一行都不用改，系统照常开工。

干过软件的人都知道这件事有多反常。随便哪套系统，最中间总是最难动的地方。订单主流程、登录模块、那段谁也不敢碰的历史代码，改一次，别的模块跟着抖三抖。中心就该是焊死的，这是常识。

这套系统把常识倒了过来。倒过来的理由、做法、代价，是这本书要讲的事。

这本书在讲什么

大模型的本事大家都见过了。写代码、读文档、讲道理，问一句答一句，答得像模像样。

可让它真去干活，事情就不对了。读一个文件，跑一条命令，改错了改回来，干到一半断了明天接着干。这些活，模型自己一件都够不着。它会想，不会动手。

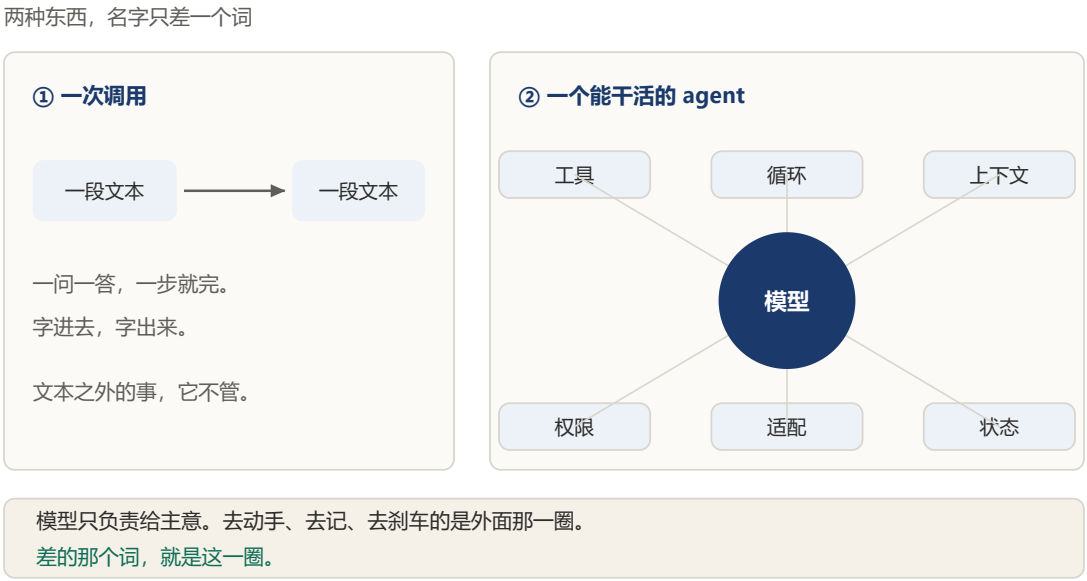


图 1：模型还是同一个模型，能不能干活全看外面那一圈〔本书〕

左边是大多数人熟悉的用法：话进去，话出来，中间什么都没有。右边才能把活干成：模型还在正中间，外面围上了一整圈人马，管动手的、管记事的、管把关的、管定下一步的。同一个模

型，接上这一圈就能干活，不接就只会聊天。

这一圈就是这本书的对象。它有个名字，叫 harness，书里叫它外壳。模型几个月换一代，快得来不及追；这一圈里沉淀的东西，却几年都不用动。谁把这一圈想清楚，谁就不用在每次模型换代时从头再来。

全书地图

这本书分六部，按从外到里的顺序走。

第 1 部进门，说清楚这件事是什么、难在哪，为什么值得专门写一本书。第 2 部先看一眼，把整栋楼拆开，六层各管什么，先给地图不给理由。第 3 部拆内核，五个房间逐个进，每进一个讲清一条设计主张。第 4 部走一遍，跟着一句问话、一次工具调用，看这些东西怎么转起来。第 5 部碰边界，讲它在哪里会塌，出过什么事，代价是什么。第 6 部收，把能带走的判断留下，别的放下。

怎么读这本书

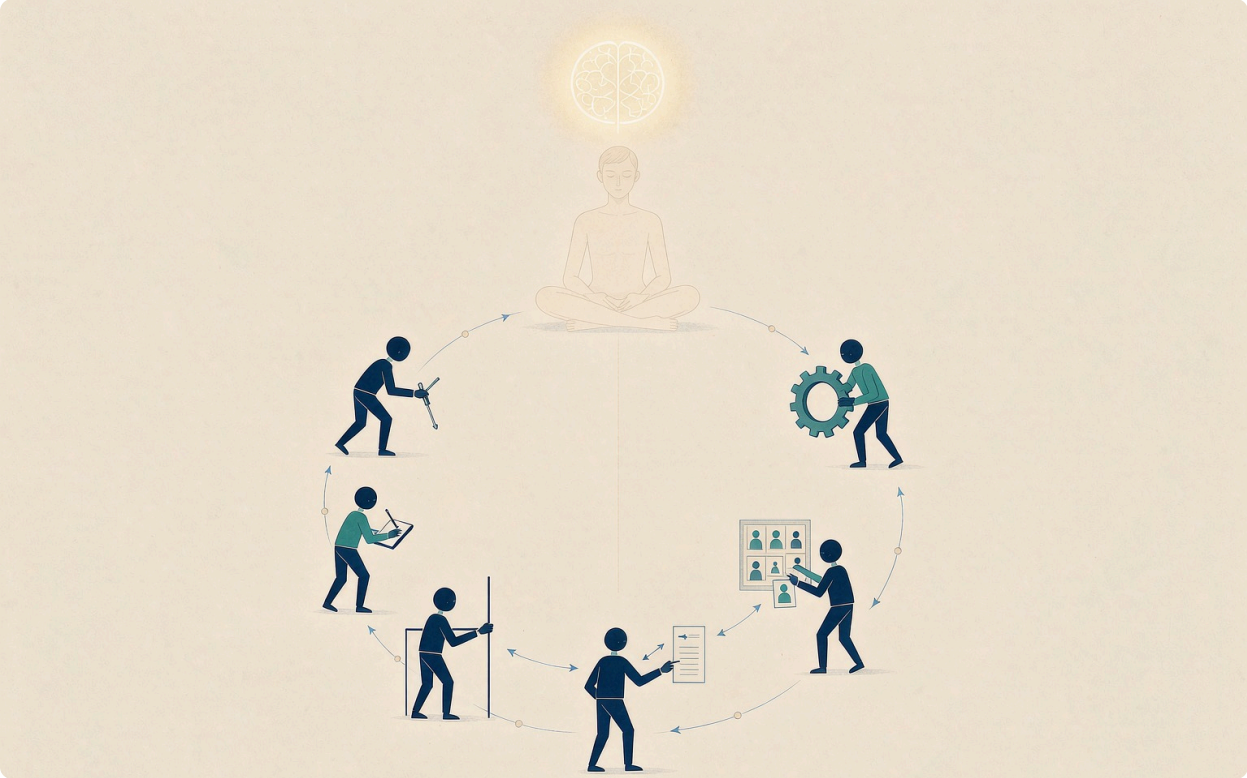
每一节都分两层。第一层讲人话，用师傅带队伍干活的故事把设计讲完，一个术语都不出现。第二层上术语，正式名字、机制、边界、代价，给要去自己动手的人。

只想知道个大概，读第一层就够。要拿去用，两层都读，再翻翻节末的对照表和易错点。

书里的类比全书通用：师傅的脑子是模型，一整套人马是外壳。第 1 部会把它从头讲一遍，后面各节直接用，不再解释。

那么，回到开头那件反常识的事。最中间那层凭什么能整块抽走？答案在第 3 部。但先得进门，看看这一圈到底在管什么。

第 1 部 · 进门



差的不是师傅的脑子，是外面那一圈人马。

§01 模型明明会写代码，为什么还是干不成活

现在的模型，代码写得比很多熟手都快。给它一段报错，它能说出病因；给它一个需求，它能写出像模像样的实现。

可把这个模型接进日常开发，让它真去干活，事情立刻不对劲。它说「我先看一下配置文件」，然后就没有然后了。它说「我把测试跑一遍」，终端里什么都没有发生。它说得头头是道，手上空无一物。

这不是模型不够好。这一节要讲的是：差的从来不是那颗脑子，是脑子外面那一圈。照例分两层，先讲人话，再上术语。

1.1 小白版：师傅站在门口，进不了厨房

家里厨房的灯不亮了，叫个修灯师傅。师傅来了，站在门口，隔着门喊：「先断电，再拆开开关看看，可能是线松了。」

说得全对。可他自己没有手。够不着开关，拧不动螺丝，连电闸都拉不下来。喊得再对，灯也不会自己亮。

AI 就是这么个师傅。它的脑子极好，知道每一步该怎么走，但它自己一步也走不了。想让它把灯修好，得给它配一整套人马：拧螺丝的，照它说的动手；记笔记的，把干过的事记下来，免得干到一半忘了前面；把门的，它说要砸墙的时候拦一句「这面墙能不能砸」；领班的，看情况、决定、动手、看结果、再决定，把整件事的节奏串起来。

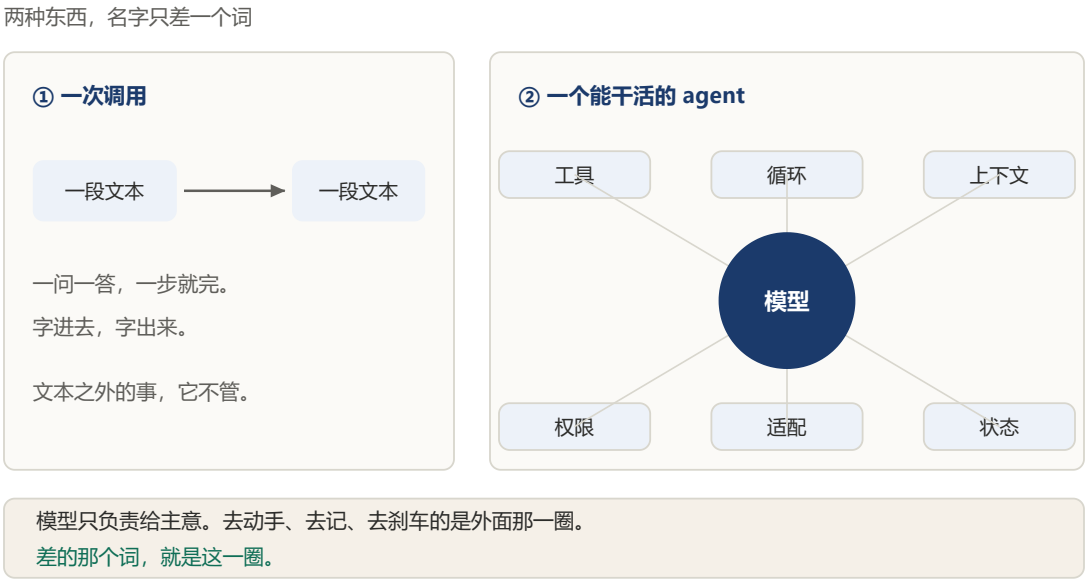


图 1：师傅还是同一个师傅，配上人马才能干活〔重印自 §00〕

左边是只有脑子的师傅：话进去，话出来，灯还是黑的。右边是配上人马的师傅：人马围着他转，他说一句，有人动一次手，有人记一笔账，有人把一道关。灯就是这样亮的。

注意一件事：左右两边是同一颗脑子。差别全在外面那套人马。所以「它会不会干活」这个问题，问脑子是问不出答案的，得看人马配得怎么样。

这套人马要干的活，数下来是六件。跟着师傅的吩咐一连走好几轮，不能走一轮就散伙；真去动手，读东西、改东西、把该跑的东西跑起来；哪些话该记着哪些话该丢，心里有数；危险的活先拦下来问一句；干过的事留底，明天来了能接着干；师傅要是换了人，队伍不用重招。

六件活，没有一件是师傅自己能办的。件件都得人马里有专人顶上。

1.2 专业版：外面那一圈要管的六件事

这套人马有正式名字，叫外壳（harness）。它是模型和真实世界之间的那一整层，模型出主意，它把主意变成行动。



图 2：外壳上接模型、下接真实世界，模型的每句话都要经过它才能落地〔本书〕

图上中间那个框就是外壳。模型在上面，只产出文字；真实世界在下面，只认动作。中间这层的职责，拆开来是六件事。

第一，agent 循环。维持多轮来回：观察现状、决定下一步、执行、读结果、再决定。模型只负责其中「决定」那一拍，其余各拍都是循环在走。

第二，工具执行。模型说「运行测试」，得有人真的去执行这条命令，并把输出带回来。读文件、写文件、跑命令，都归这件。

第三，上下文管理。模型的记性有上限，哪些历史该留下、哪些该丢掉、留下的怎么排，直接决定它下一步的判断质量。

第四，权限与沙箱。模型的提议不等于许可。要删掉整个目录的命令，得先被拦下来，按规矩问一句。

第五，会话状态。做过什么、做到哪一步，要落在模型之外的地方。任务中断后接着干，靠的不是模型的记性，是这份记录。

第六，模型适配。换一个模型，调用方式、返回格式、能力边界都不一样。这层差异该由外壳吸收，不该漏进业务逻辑。

六件事有一条共同的性质：模型一件都办不了，它们全都要由代码去接。这就是「外壳」存在的理由，不是锦上添花，是缺了它模型就只剩一张嘴。

适用边界也要说清。外壳不是越厚越好。外面那一圈每厚一分，读懂它的成本就高一分。厚度必须换来职责清晰：六件事各归其位，谁管什么一眼看得见。厚厚一圈却说不清楚哪件事归谁管，那是最坏的结果，比没有外壳还难维护。

1.3 两种说法的对照

本节新出现的说法，对应关系如下。全书通用的那套类比（师傅的脑子、一整套人马这些），到第 3 部会交一份完整对照表，这里只收本节的新词。

小白说法	专业术语
话进去，话出来	单次调用（single call）
跟着吩咐一连走好几轮	agent 循环（agent loop）
干过的事留底，明天接着干	会话状态持久化（session persistence）
师傅换了人，队伍不用重招	模型适配（model adapter）

1.4 最容易理解错的地方

第一处：把干不成活归咎于模型不够聪明。错。同一个模型，接上外壳就能干活，不接就只会聊天。瓶颈在外面那一圈，不在脑子。

第二处：以为把六件事写进提示词就行了。错。提示词能告诉模型该怎么做，但读文件、拦危险命令、断点续干这些动作，提示词一件都执行不了。它们必须由代码完成，这是提示词和外壳的分界。

第三处：以为外壳是模型的附属品，模型升级外壳跟着换。恰好相反。模型几个月换一代，外壳里沉淀的东西几年都不用动。设计得好的外壳，换模型像换师傅，队伍还是那支队伍。

1.5 这条能搬去哪

一个自查动作。随便找一个号称能干活的 AI 产品，对着六件事逐项对：多轮来回谁维持，动手谁执行，记性谁管，危险动作谁拦，记录谁留，换模型怎么办。答不上来的那几项，就是它干不成活的地方。

反过来也成立。评估自己手里的项目，看这六件事散落在哪。如果它们混在业务逻辑里，每一处模型调用都各管各的，那就是还没有外壳，只有一堆散件。

§02 harness 到底是什么，和框架有什么不一样

上一节说，差的不是模型，是外面那一圈。那一圈有个名字，叫 harness。

这个名字值一节。因为圈子里用得很乱。有人管它叫框架。有人管它叫脚手架。还有人干脆叫它「那个命令行工具」。名字乱了，谈设计就谈不下去。这一节把词义钉死，顺便把最容易混淆的三个邻居分开。还是先讲人话，再上术语。

2.1 小白版：马具、材料店和成品房

harness 这个词，本来是马具。

套在马身上的一整套皮带和挽具。马只会跑，不会拉车。要让它变成能拉货的牲口，得把它的劲引到车上去。干这件事的就是马具。

马具有个很妙的地方：它不改马。马还是那匹马，脾气、脚力一点没变。它改的是马使出来的劲往哪儿去。软件里借这个词，借的就是这层意思。外面那一圈不改师傅的脑子。它安排的是脑子的力气怎么落到活上。



图 2：人马不改师傅，只把师傅的力气引到活上〔重印自 §01〕

师傅在上面出主意，活计在下面等结果。中间这套人马就是马具。主意从上面下来，动作往下面去，来回都得过它。

弄清了马具，再看它最容易被认错的三家店。要盖房，街上能买到三种东西。

第一家是材料店。木料、砖头、水泥，附一本说明书。它很客气：要什么拿什么，怎么盖随买主便，它绝不插手。第二家是图纸铺。地基打好了，墙的位置画好了。空架子立在那儿，血肉自己填，填完架子就拆。第三家是成品房。拎包入住，但房子有主见。门禁早就装好了，不刷卡门都别想进。装不装门禁这种事，它根本不问。

三家都自称帮人造房，脾气完全不同。材料店把决定权全交出来，出了事买主自己扛。图纸铺给个头，尾自己收。成品房把自己的规矩变成住户的规矩，住进去就得照它的来。外面那一圈是第三种：它不是一堆零件，是一套带着规矩的成品。把门的说不行，就是不行，没有商量。

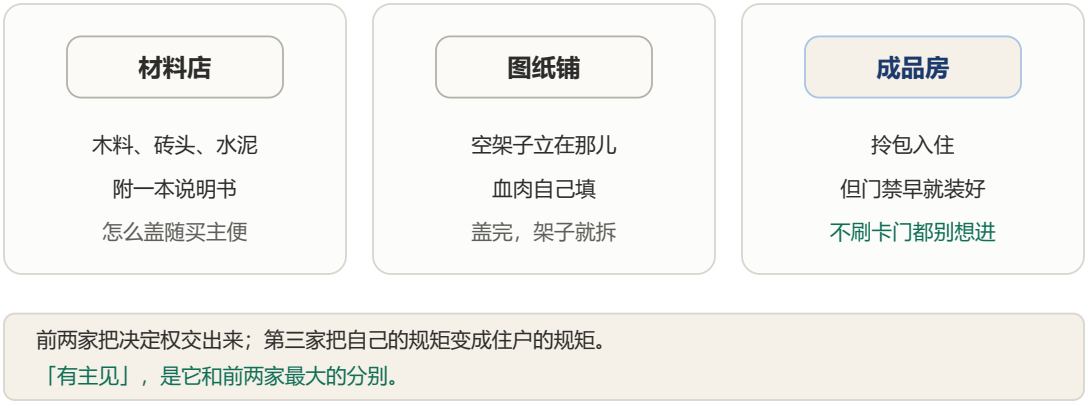


图 3：三家都自称帮人造房，脾气完全不同〔本书〕

前两家把决定权交出来，第三家把自己的规矩变成住户的规矩。这个「有主见」，是它跟前两家最大的分别，也是后面所有故事的根。

2.2 专业版：行业公式、核心不变式与三条分界

行业里有条半开玩笑的公式：Agent = 模型 + 外壳。配套的调侃是：如果你不是模型，你就是外壳。玩笑归玩笑，这条公式把讨论对象钉住了。模型之外、为了让它干活而存在的一切，统称外壳。

外壳有一条核心不变式。模型提议，外壳授权、执行、记录、恢复。模型产出的是建议，不是事实；要不要放行、真的去做、做完了留底、断了能接回来。这四拍，全在外壳手里。不变式的意思是：无论哪家实现、换什么模型，这四拍的归属不许变。

再说三条分界。外壳和框架的分界，判据只有一条：谁的入口先跑。用框架，程序从自己的入口开始跑，在需要的位置调用框架；用外壳，程序从它的入口开始跑，自己写的东西挂进去等它调用。控制流方向相反。框架的扩展是调出来，外壳的扩展是挂进去。

外壳和脚手架的分界在寿命。脚手架服务于搭建期，房子盖完就拆。外壳是长期住户，只要系统还在干活，它就一直在场，每一次提议都要过它。

外壳和两者的共同分别是有主见。框架和脚手架都是谦让的，给材料、给图纸，不替人做决定。外壳是有主见的成品房。它不跟住户商量「要不要权限校验」。规矩直接立在那，不校验门都别想进。这份强制不是缺陷，是它敢把模型接到真实世界上的底气。

外壳里装着哪几样东西？就是上一节数过的六件。agent 循环、工具执行、上下文管理。权限与沙箱、会话状态、模型适配。六件凑齐，模型的提议才能走完授权、执行、记录、恢复这四拍。少一件，不变式就破一个角。

边界和代价也要摆出来。成品房的代价是迁就。它的规矩不一定合所有人的意，不认同就得绕开或者搬走。有主见换来的安全，是用让渡自由付的账。这笔账划不划算，要看接的是什么样的活。这也是下一节三家做法分歧的起点。

2.3 两种说法的对照

本节新出现的说法，对应关系如下。

小白说法	专业术语
材料店，自己盖房	框架（framework）
图纸铺，空架子盖完就拆	脚手架（scaffolding）
有主见的成品房	外壳（harness）
不刷卡门都别想进	强制性约束（默认开启、不可绕过的校验）
主意从上面下来，动作往下面去	模型提议，外壳授权、执行、记录、恢复

2.4 最容易理解错的地方

第一处：把 harness 当成框架的洋名字。错。两者的控制流方向相反，判据是程序从谁的入口先跑。名字像，脾气相反。

第二处：以为外壳是模型的配件，主角是模型，外壳随便搭搭就行。错。核心不变式里，授权、执行、记录、恢复四拍全在外壳手里。模型只占提议一拍。谁是主角，看拍数就知道。

第三处：以为有主见是缺点，是不灵活。错。不校验门都别想进，恰恰是外壳敢把模型接进真实环境的理由。谦让的框架把这份责任留给了使用方。有主见的外壳把它收进了自己手里。是分工不同，不是高下之分。

2.5 这条能搬去哪

一个自查动作。拿一个常用的库或平台，问一句：程序从谁的入口先跑？从自己的入口跑、需要时调它，那是框架；从它的入口跑、自己的代码挂进去，那是外壳。再补一句：它的规矩能不能绕过？能绕过的是材料，绕不过的才是成品房。两问下来，手里的东西姓什么就清楚了。

§03 同样是外壳，为什么三家造出三种东西

外壳该管什么，前两节说清了。照理说，管同六件事，各家造出来的东西应该差不多。

实际上差得很远。市面上有三家有代表性的外壳，摆在一起看，像是三个行业的产品。这是全书唯一一节横向对比，看清这三家的分歧，才能看懂这本书的主角为什么长成那个样子。还是先讲人话，再上术语。

3.1 小白版：三家装修公司接同一个活

同一条街上三家装修公司，接的是同一个活：给师傅配一套人马，让他能把活干成。

第一家卖成品房。人马齐备，规矩定好，拎包入住。今天签约，明天师傅就能开工。代价是格局改不了：墙不能砸，家具就这些，想换一种带队伍的方式，只能等房东出新户型。

第二家卖图纸。房子长什么样它不包办，但水电煤从哪进、管线的口留在哪，图纸上标得清清楚楚。照这个口，接谁家的管子都行。它卖的不是房，是那个口。想要自由，自己得懂行。

第三家卖插座。房子里几乎是空的，只有一排排标准插座。冰箱、洗衣机自己带，插上就能用。最特别的是，连管家都是插上去的，看他不顺眼，拔下来换一个。空到了家，也自由到了家，但置办家当的活，全落在自己头上。

三家的必做活是一样的。水电得通，门得能锁，师傅干了活得有地方记账。这些活谁家都不敢省，也都做下来了。拉开差距的是选做活：房子留不留得动、师傅换不换得了、账本让不让拿回家自己看。成品房说不必，图纸说可以商量，插座说全都随你。



图 4：必做活三家都做，拉开差距的是选做活〔本书〕

所以这不是一场谁好谁坏的比赛，是三种卖法。看清卖法，比争出高下有用得多。

3.2 专业版：八维度对比与两种考题

三家各有名字。卖成品房的是 Claude Code，卖图纸接口的是 Codex CLI，卖插座的是这本书的主角 DSH。

先交代背景：三家的功能面已经收敛。记忆与指令文件、分层配置、外部能力接入、沙箱加审批、无界面运行、企业托管，这些三家都有。比的不再是功能有无，而是同一圈事上的取舍。真正的分歧在下面八个维度。

维度	DSH	Claude Code	Codex CLI
开源	开源（MIT）	闭源	开源
架构	插件内核	垂直集成	垂直集成
哲学	一切皆插件	开箱即用	Protocol-first
模型	任意兼容端点	仅自家模型	仅自家模型
会话日志	本地明文，仅追加，可分叉可重放	托管	托管
运行时自扩展	支持热加载	不支持	不支持
默认沙箱	只读，故障安全	权限提示	权限提示
已收敛功能（记忆文件、分层配置、托管等）	有	有	有

表里的分歧可以归成一行：Claude Code 卖成品，Codex CLI 卖接口，DSH 卖插座。成品房把决定都替人做好了；接口把接法规定好，剩下的自己接；插座连管家都是插上去的，房子里几乎不留固定岗位。

换个角度看，这八行是两份考卷。一份是必答题：循环、上下文、工具接入、会话持久、权限沙箱。这五道三家都答了，答案大同小异，这是行业的及格线。另一份是选答题：内核要不要空置、模型要不要插件化、日志要不要可重放。这三道，只有 DSH 这么选，另外两家选择了把答案收在自己手里。

选答题的关键在代价的走向。DSH 的答案不是更好，是把复杂度从实现方转移给了使用者。内核空置，意味着读一遍代码看不出系统在干什么，行为要从配置里重建；模型插件化，意味着接什么模型自己负责；日志可重放，意味着保管和利用日志是自己的事。成品房把这些麻烦吞下去，住户省心；插座把这些麻烦交出来，住户自由。谁付这笔账，是三家真正的分歧。

3.3 两种说法的对照

本节新出现的说法，对应关系如下。

小白说法	专业术语
卖成品房	垂直集成、开箱即用（Claude Code）
卖图纸，管线的口标清楚	Protocol-first，卖接口（Codex CLI）
卖插座，管家也是插上去的	一切皆插件、内核空置（DSH）
必做活	必答题：循环、上下文、工具接入、会话持久、权限沙箱
选做活	选答题：内核空置、模型插件化、日志可重放

3.4 最容易理解错的地方

第一处：把三家对比看成功能竞赛，数谁家功能多。错。功能面早已收敛，该有的三家都有。真正的分歧是复杂度由谁付，这在功能清单上看不出来。

第二处：以为选答题答得多就是更先进。错。选答题是取舍题，不是加分题。内核空置换来替换自由，付出的是可读性；开箱即用付出的是改造空间。两种付法对应两类用户，没有高下。

第三处：以为开源和插件化是一回事。错。开源说的是看不看得到、改不改得了源码；插件化说的是换一样能力动不动主体。三家在两条轴上的位置各不相同，不能混成一条。

3.5 这条能搬去哪

一个自查动作。评估任何一个 agent 产品，先把它答的卷子拆开：必答题（循环、上下文、工具接入、会话持久、权限沙箱）答得齐不齐，不齐的直接淘汰；再看选答题，问一句：复杂度是实现方吞了，还是留给了使用方。想清楚自己愿意付哪笔账，选型就完成了一大半。

§04 难在哪：能力在变、模型在换、环境不可信、任务还老断

前三节把对象说清了：模型外面那一圈，叫外壳，外壳里要管六件事。听上去，找几个人把这六件事办妥，不就完了？

开头往往真是这么顺。队伍刚拉起来，家伙就那几样，师傅就那一位，出了岔子收摊重开。三个月以后开始难受。这一节讲四道绕不过去的难处，只讲难处，不讲解法。解法是这本书后面五部的事。

4.1 小白版：队伍开张三个月以后

人马配齐了，师傅有人伺候了，头一个月跑得顺顺当当。然后四件事一件一件找上门。

第一件，活计的花样老变。今天修灯，明天修水管，后天有客户要自带一套进口家伙。每添一样家伙，领班的就得学一遍什么时候递它。

第二件，师傅老换。这行里师傅换得勤，几个月一茬。新师傅脑子一样好，可说话路数、做事习惯全不一样，人马得重新磨合一遍。

第三件，干活的地方不可信。师傅照着印象说「把里屋那扇窗打开」，里屋根本没有窗。更险的是，一句听着平平常常的吩咐，照着做能闯大祸。师傅不是使坏，他是真不知道这房子的边界在哪。

第四件，活又长又老断。一桩活一干大半天，中间会断电，干活的会被人临时叫走，记笔记的本子会写满。断了不要紧，怕的是接不上：明天来了，谁也不记得昨天干到哪。

自然的应对是领班的拿个小本，记满「碰到什么该怎么办」。这个办法撑不久。每添一样活计就得添一条，条目多了，谁也说不清哪条管哪条，改一条碰倒三条。再进一步，立一套正式的章程，家伙们都照章程打造。这步对，但不够：章程只管怎么动手，没管什么时候动手、砸了算谁的、不用了怎么请走。



图 5：四道难处，道道绕不过去〔本书〕

左栏这四道难处，没有一道能靠师傅更聪明来解决。右栏的应对先不展开，它们各自占着这本书后面的一整部。这一节只需要记住左栏。

4.2 专业版：四道难题的性质与排序

把人话换成术语，四道难处各有名字。

第一道，能力集合持续变化。工具从几样涨到几十样，还要接私有系统。能力每变一次，写死在循环里的调用就得跟着改一次。

第二道，模型更换。换代以月计，没有公司愿意绑死在一款模型上。各家的调用方式、返回格式、能力边界都不一样。换模型不是换一个地址，是重新适配一层。

第三道，环境不可信。模型会照着上下文编出一个不存在的路径，也会把一条看似无害的指令执行成灾难。它不是恶意，是不知道边界在哪。不知道边界，恰恰是最需要边界的情形。

第四道，长任务与中断。真实任务可能跑上几十分钟。中间网络会断，进程会被终止。上下文也会超出模型的记忆上限。断点续干不能指望模型的记性，它的记性是租来的。

硬撑为什么撑不住，值得说透。第一层硬撑是在循环里写分支，遇到什么调什么。每加一种能力就改一次循环。改的地方越多，改错的机会越多。改到后来，没人说得出循环的行为边界。第二层是抽接口、定规范。这一步是转折点，但它只回答了「怎么调用」。什么时候调用、失败算谁的、怎么拿掉，这三问它没答。后面三个问题，才是外壳真正的功课。

四道难题不并列，性质和后果差得很远。能力变化和模型更换是结构问题，决定这套东西能不能长大。环境不可信是安全问题，不常出，出一次可能收不住，决定这套东西敢不敢交出控制权。任务中断是持久化问题，决定这套东西能不能干长活。这本书的顺序也照这个轻重排：先讲结构，再讲边界，收尾讲怎么带走。



图 6：目标不是让左边不变，是左边变的时候右边不动〔本书〕

左边的事一定会发生：团队换人写法就变，模型出新版就得升，业务收紧权限就得调。右边的事不该被牵着走。定下的规矩、留给人看的记录、别人接进来的方法。这些每动一次都要付代价。让左边不变是做不到的，能做的只有一件事：左边变的时候，右边不动。这句话是整本书的判据，后面每条设计都能拿它验一遍。

难处摆完了。怎么让右边不动，是第 2 部开始的事。

4.3 两种说法的对照

本节新出现的说法，对应关系如下。

小白说法	专业术语
活计的花样老变	能力集合持续变化
师傅老换，人马重新磨合	模型更换与适配
师傅编出一个不存在的房间	幻觉（hallucination）
本子写满了	上下文超出记忆上限

4.4 最容易理解错的地方

第一处：把四道难题当成并列的四件事，平均用力。错。它们性质不同：两道是结构问题，一道是安全问题，一道是持久化问题。安全问题不常出，出一次收不住；结构问题天天在付利息。用力顺序错了，投入全打水漂。

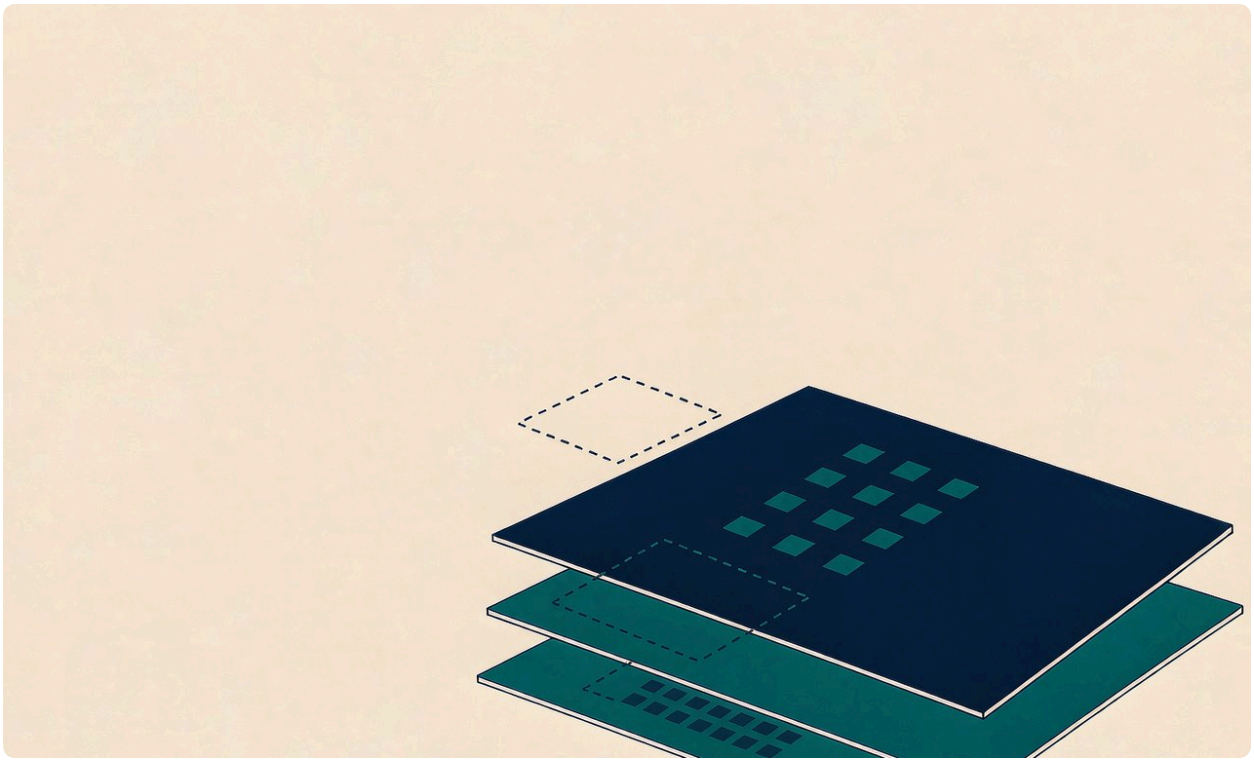
第二处：以为抽了接口、定了规范，难题就解决了。错。接口只回答怎么调用。什么时候调用、失败算谁的、怎么拿掉，这三问没有答案，难题一道都没少。

第三处：把设计目标定成「让会变的不再变」。错。左边那些事一定会发生，拦不住。目标是左边变的时候右边不动，不是左边不动。定错目标，所有努力都用在堵窟窿上。

4.5 这条能搬去哪

一个自查动作。找一个手头的项目，挑出最近加进去的一样能力，数一数为了加它动了几处地方。只动了登记的那一处，是及格；判断流程里、拦截规矩里、记账的地方各改一句，就是动了三处。改的处数，就是这个项目在四道难题上的成绩单。

第 2 部 · 先看一眼



先给地图，不给理由。

§05 拆开它，六层各管什么

这套人马到底有多少人、怎么站？打开看，是一栋楼，六层。

这一节只干一件事：发楼层指引。哪层叫什么，哪层管什么，谁挨着谁。为什么这么切、切错了付什么代价，是下一部的事。先把地图拿到手，理由后面再补。

5.1 小白版：公司的六层楼

进一家陌生的公司，门房递来一张楼层指引。从下往上看。

地下室是总部最里面那一层。放的是换不出花样的东西：几点开工、本子往哪放、出了事留不留字条。哪家公司都差不多，所以没人去动它。它小，但整栋楼都压在它上头。

一楼是干活的班组。拧螺丝的、跑腿的、接外线的，全在这层。多雇一个班组，公司多一样手艺，地下室不用跟着动。检验也简单：加一样手艺要不要翻修地下室？要，就是楼盖错了。

班组旁边是安检口，比别的层都小。它的活只有一件：哪些事不许干。怎么把活干成，不归它管。刹车片从来比发动机小，这层最小，天经地义。

再往上是门房，对外的窗口。有人从网页来，有人从桌面来，有人从黑窗口来，还有别家的程序来敲门。四种来法，都从这一个窗口进。门房只管接待传话，从不替班组拿主意。

顶楼最小，只坐一个人：写排班表的。今天雇谁、用哪套编制，他一张纸写完就下班，别的一概不管。

最底下还有半层杂物间，堆着试用的家伙。看楼时可以略过，里面的东西下个季度可能就扔了。



图 7：六层楼，每层只回答一个问题〔本书〕

图里的箭头不是谁给谁打电话，是谁能换掉谁。越靠上的层，定得越晚，换起来越省事。别读反了：位置靠上不等于重要，只等于更能改。



图 8：四种来法都从门房进，门房不拿主意〔本书〕

窗口能加，地下室不必跟着长，这是楼层画对了的样子。画错了有典型症状：换一种接待方式，班组怎么干活就得跟着改。真到那一步，说明主意是门房拿的，越界了。

5.2 专业版：六层的名字和边界

六层的正式名字，从上到下：装配层、宿主层、内核层、能力层、边界层、支撑层。

各层的职责边界，一层一句话。

装配层只回答「这次装什么」。一份名单，不写业务，不持有状态。它小的理由是替换语义：凡是会按形态取不同值的东西，都不能住在这一层。

宿主层只回答「人在哪儿用」。四种客户端入口共用同一套内核。它是六层里工程量最大的一层，却一个决策都不做。

内核层是一次任务绕不开的几件东西。哪几件、为什么偏偏留下它们，第 3 部专门用一节讲。这里只给判据：删掉它，一次任务就进行不下去。

能力层是挂在循环下面的一排插孔，装哪个，agent 就多哪只手。边界层只回答「哪些事不能做」，策略、审批、受限执行都在这层。支撑层是试验田和杂物间，看架构时可以先剔掉。

哪层是楼的中心？数代码会数走眼。工程量最大的宿主层排不进中心，真中心是那个被指着最多的。依赖箭头指向谁，谁才是内核。

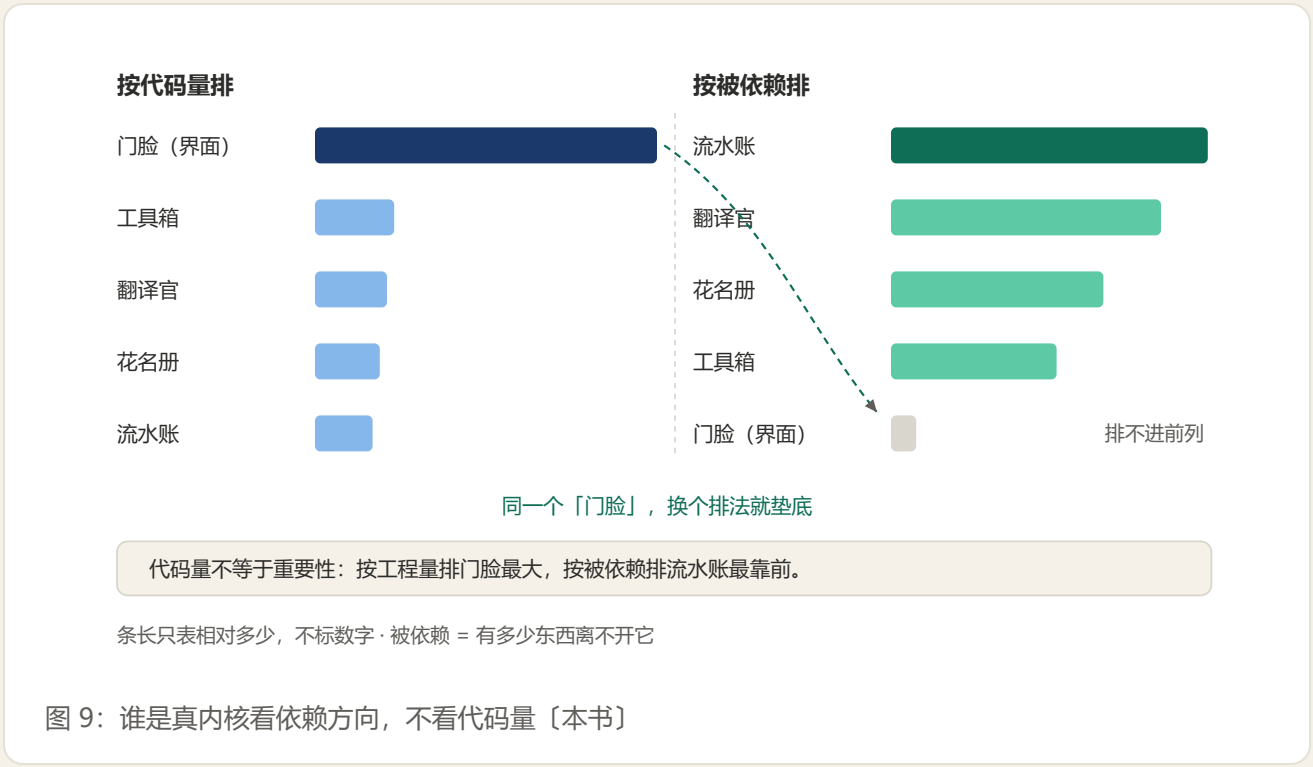


图 9：谁是真内核看依赖方向，不看代码量〔本书〕

两种排序完全不同：按工程量排，门脸最大；按被依赖排，记笔记的最靠前。代码量不等于重要性，出处就在这里。判断自己项目的内核也一样：别看哪个文件最长，看删掉它，多少东西活不下去。

这节只给地图。每条边界为什么这么画、画错一层付什么代价，第 3 部一层一层拆。现在只带走一句：每层只管一件事，不越界。越界的代价是同一个，本该只换一层的改动，蔓延成改多处。

5.3 两种说法的对照

小白说法	专业术语
六层楼	六层分层
地下室、总部最里面	内核层（kernel layer）
干活的班组	能力层（capability layer）
安检口	边界层（boundary layer：策略、审批、受限执行）
门房、对外窗口	宿主层（host layer）
顶楼写排班表的	装配层（assembly layer）
杂物间	支撑层（support layer）
谁指着谁	依赖方向

5.4 最容易理解错的地方

第一处：把位置靠上理解成更重要。靠上只说明定得晚、换得动。重要性看依赖方向，不看楼层高低。

第二处：把代码量最大的层当成核心。宿主层工程量最大，却一个决策不做。真内核是被指着最多的那层，往往小得不起眼。

第三处：把六层当成标准答案照抄。层数不必抄，判据才要抄：每层只回答一个问题；换掉层里一件东西，要动别层，就再切一刀。

5.5 这条能搬去哪

两个自查动作。

第一，翻出项目的入口，数一数几行在说「装什么」，几行在说「怎么干活」。两种话混在一处，装配层就没从别的层里分出来。

第二，给现有模块各写一句话：这层只回答什么问题。写不出来的那个，多半是两层的活挤在了一层，切层的第一刀就落在它身上。

§06 同一套东西，为什么能装出四种形态

同一家公司，周一全员到岗，周三只来三个人，月底还跟了一队实习生。楼是同一栋，人是同一批。

这套人马也一样。同一套东西，装出四种形态，差别只有一条：今天雇了哪些临时工。

这一节讲装的三样东西：规矩怎么叠，编制怎么配，哪些位子备了替补。

6.1 小白版：规矩叠三层，编制有四套

公司里的规矩是三层叠着来的。

公司定一份大规矩：几点上班，假怎么请。部门再定一份小的：这个组周五不打卡。工位上还能贴张纸条：这星期先走一步。三层撞了听谁的？听最小的。纸条压过部门，部门压过公司。不是纸条官大，是它最晚贴、离人最近。大规矩是兜底，兜不住的才轮到纸条。



图 10：规矩三层叠着来，小的压大的〔本书〕

越晚贴上的越说了算，顺序就是优先级。层数低不等于权力大，这条最容易记反。同一句话被写了三遍，作数的只有最晚那一遍。

规矩定了怎么干活，编制定今天来多少人。四套编制：满编，什么都带；带徒弟，老师傅干活，边上跟个学的；轻装，只带最要紧的几样；招新，摊子铺开，专门迎接刚入行的。

四套编制不是四家公司。楼是同一栋，脑子是同一颗，差别只在排班表上勾了哪些临时工。满编勾得最多，轻装勾得最少。

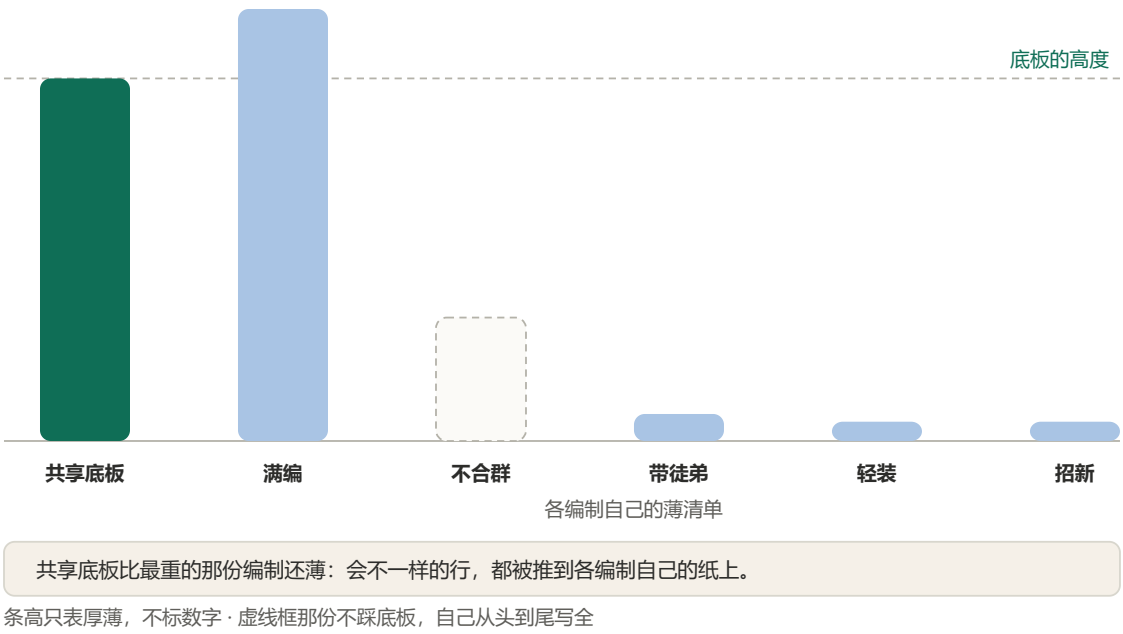


图 11：四套编制共用一块薄底板，差别只在多雇了谁〔本书〕

公用的底板比想象中薄。凡是各编制不一样的东西，都不写进底板，推到各编制自己的纸上。底板越薄，换编制越像换一张纸，不是翻修一栋楼。也有不合群的：有一份编制干脆不踩底板，自己从头到尾写全。它不是叛逆，是另一份并列的排班表。

第三样：哪些位子能换人。

不是每个位子都有替补。有的位子上贴着「可替换」，还真备着第二个人选。有的位子也是临时工，可全公司就他一个会干，他请假就停摆。位子是当初特意留的，替补是后来慢慢补的。这是两笔账，别算成一笔。

6.2 专业版：三个角色、一条顺序、四种 preset

装配的正式说法是三个角色，各管一段，互不重叠。

角色	它是什么	它说了算什么
profile	一份名单	装什么
bundle	可分发的零件包	默认值
patch	按名字定位的修改条	最终值

三者不能合成一个：名单回答装什么，零件包回答默认给什么，修改条回答最终是什么。混在一起，形态就没法只换一层。

叠放的顺序，一句话说得完：

> 各层按此顺序应用在空条目列表之上：先按 profile 列出的顺序应用每个组合包，然后是 profile 的补丁文件，然后是 home 级的那份。

从空名单起叠，同一行被写下好几次，生效的只有最晚那次。修改条命中后交出的是一整块配置，系统整块替换目标行，不做逐字段合并。不做合并的理由是歧义：数组算追加还是替换、嵌套算到第几层，每条都得单独定规则，规则一多，结果没人敢保证。宁可让人多抄几行，换任何一行只有一个出处，整棵树能原样打印。

四种 preset 形态：标准、PTC、极简、创造。它们是四份并列名单，多数以同一块共享底座开头，再加一层薄清单，只回答「我哪里跟别人不一样」。有一份不套底座，自己就是完整的一棵树。机制里没有父子名单这个概念，形态之间并列，不记继承。

「能换的位子」的正式名字叫 seam，接缝。官方把服务分了几类：

> 服务可以是核心主干服务、可替换的能力 seam、组合包 / 组合点或独立服务。

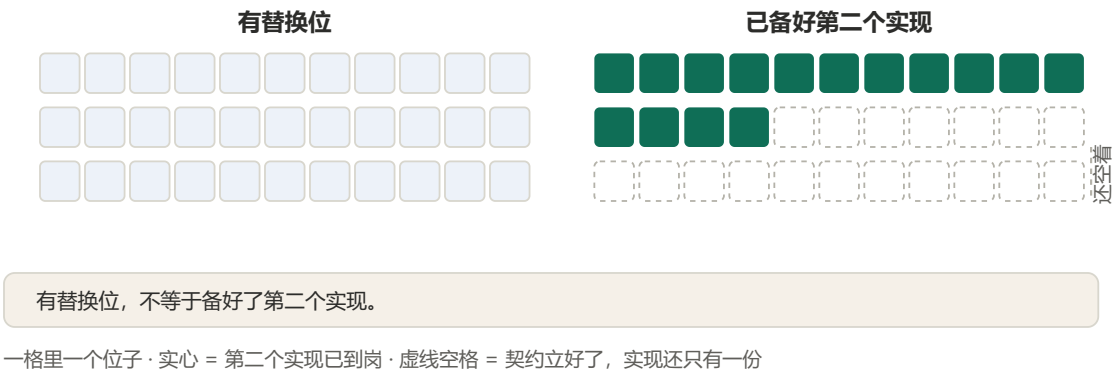


图 12：备了替补的位子，不等于替补已到岗〔本书〕

结构账和库存账是两笔账。留替换位，改的是消费方那一侧，让它们只认名字不认实现，这笔钱立契约时就付了。补第二个实现是另一笔，可以晚付，也可以别人付。空着的位子不算欠债，最贵的那笔已经付掉：消费方不再依赖具体实现。库存会动，结构不动。

这套装法的代价也摆在那：修改条必须写全，啰嗦但确定；读一遍名单看不出系统在干什么，行为得从配置里重建。省事和确定只能选一个，这里选了确定。

回到标题的问题。四种形态不是四个产品，是同一套东西按四张名单装出来的，差别只有装了哪些插件。

6.3 两种说法的对照

小白说法	专业术语
公司大规矩、部门规矩、工位纸条	三层覆盖（零件包默认值、名单补丁、用户级修改），小的压大的
四套编制（满编、带徒弟、轻装、招新）	四种 preset 形态（标准、PTC、极简、创造）
排班表上勾了谁	profile 名单
零件包	bundle
修改条	patch
备了替补的位子	接缝（seam）
位子上现在站着几个人	第二个实现是否到位（库存账）

6.4 最容易理解错的地方

第一处：以为层数低权力大。正好相反，顺序就是优先级，越晚落下越说了算，低层只是兜底。

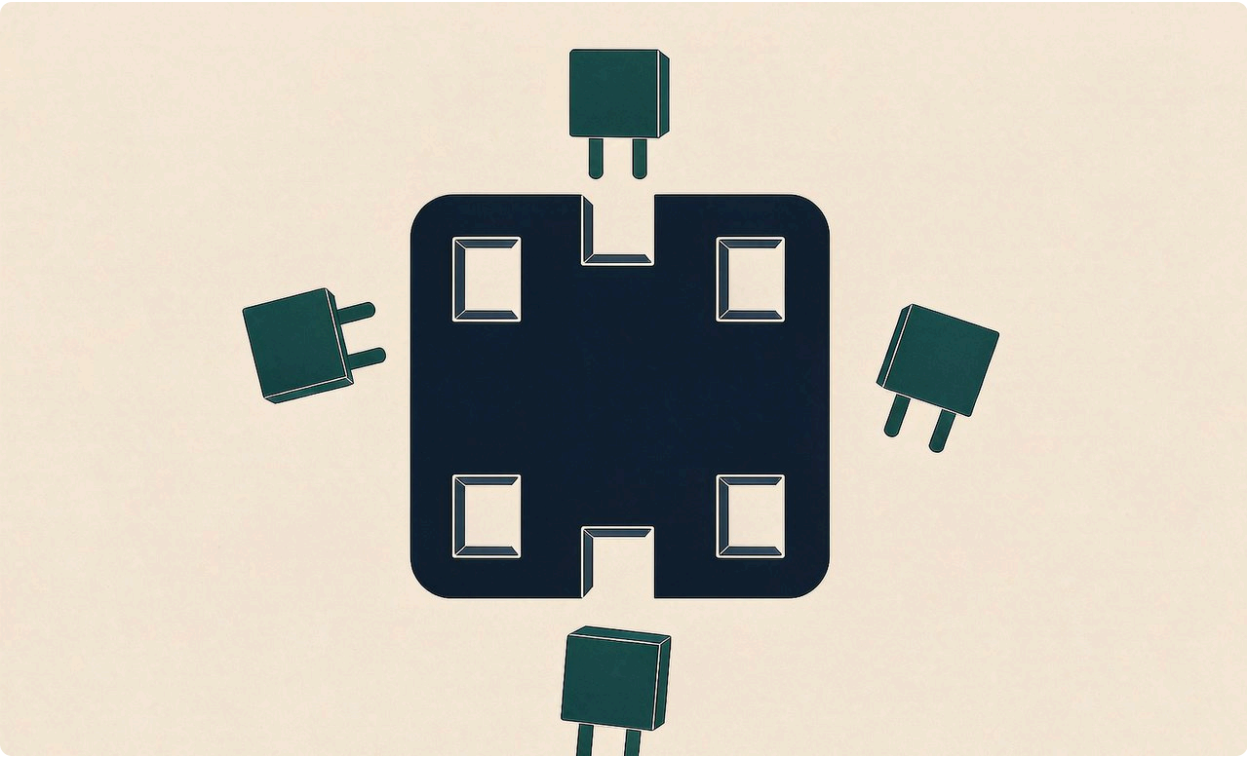
第二处：以为四种形态有父子关系，都从一个总形态继承。四份名单并列，不记父子；共用底板只是多数人的选择，有一份压根不踩底板。

第三处：把「一切皆插件」理解成什么都能换。装法统一，换法不统一。有替换位不等于备好了第二个实现，结构账和库存账得分开算。

6.5 这条能搬去哪

第一，看自己的配置系统：最终生效的那一份，能不能一条命令原样打印出来？打不出来，说明合并规则藏在实现里，打印出来的不是真相。

第二，判断一个服务能不能换，数它名下挂了几个平级实现，再看消费方代码里有没有出现实现的名字。两个问题答完，这个位子的替换半径就清楚了。



五个房间，各放一样东西。

§07 内核里是空的，连 agent 循环都是插件

每套软件里都有那么一个地方，动它之前得开会。

可能是订单主流程，可能是登录模块，可能是那段谁也不敢碰的历史代码。它未必是写得最好的部分，但一定是最难换的部分。改一次，别的模块跟着抖三抖。

DSH 里找不到这样一个地方。它最中心的那一层，可以整块拿走，别处一行都不用改。

这话凭什么成立？这一节分两层讲。第一层讲人话，第二层上术语。

7.1 小白版：一个只有脑子的师傅

家里厨房的灯不亮了，叫个修灯师傅。师傅到了，拎着一整套家伙：螺丝刀、万用表、梯子，还有个小本子。

先问一句：师傅最值钱的，是那套家伙，还是那颗脑子？

是脑子。螺丝刀谁都会拿，知道先拆哪一颗才难。

再反过来想：一个人只有脑子，没有手，修得好灯吗？修不好。他说「先断电，再拆开看看」，没人动手，等于没说。

AI 就是这么个情况。它知道该怎么干，但它自己够不着。所以想让它干活，得给它配一整套人马。



图 13：师傅只会想，这七件事得队伍里另外有人补上〔本书〕

这一整套人马有个名字，叫外壳。动手的有拧螺丝的，记事的有记笔记的，把关的有把门的，定下一步的有领班的。

中间领班最要紧。看情况、决定、动手、看结果、再决定，整个队的节奏全听他的。

大多数系统把领班焊死在公司最里面。这个选择有道理：领班换了，整个队的作风都变了，想让行为一致，就得把他攥在自己手里。代价也清楚：外面的人想要另一种节奏，只能等官方更新。等不到，就只好自己再开一家公司，重新雇一遍人。

DSH 的做法反过来。领班也是临时工，他和拧螺丝的、记笔记的站在同一层，由同一张排班表决定今天谁上。



图 14：左边领班焊在公司里，右边他和帮手站平了〔本书〕

左边想换领班，得连公司一起动。右边换领班，只改排班表上的一行。

公司最里面也不是全空着。剩下的是那些换了没意义的岗位：几点开工、本子往哪放、出了事留不留字条。这些东西哪家公司都差不多，换不出花样。凡是各家做法差别很大的，全被请了出去，一个不留。

怎么判断一个人是不是真能换？不能看他挂的工牌。有的岗位，工牌上印着「灵活编制」，可全公司每个人都直接点名找那个小张。小张一请假，全公司瘫。能用的判据只有一个：换掉他，要动几个人。

7.2 专业版：插件内核与改动半径

这套结构有个正式名字，叫插件内核。

它的核心机制是两条。

第一条，内核只留原语：时钟怎么取、日志往哪写、崩溃留不留现场。这些哪家做法都差不多，换不出花样，留着它们，系统才有个稳的底子。

第二条，其余一切都是插件。模型适配器、工具注册表、会话日志，以及 agent 循环本身，全由装配清单决定当前会话装哪些。官方的说法是：

> 产品的每一部分都是插件，包括模型适配器、工具注册表、会话日志，以及 agent 循环本身。……不存在需要打补丁的特权内核。

插件之间不互相直连。消费方按名字要能力，装配层照着清单把名字兑现成实现。所以加一样能力，等于在清单里登记一行，内核一行不动。

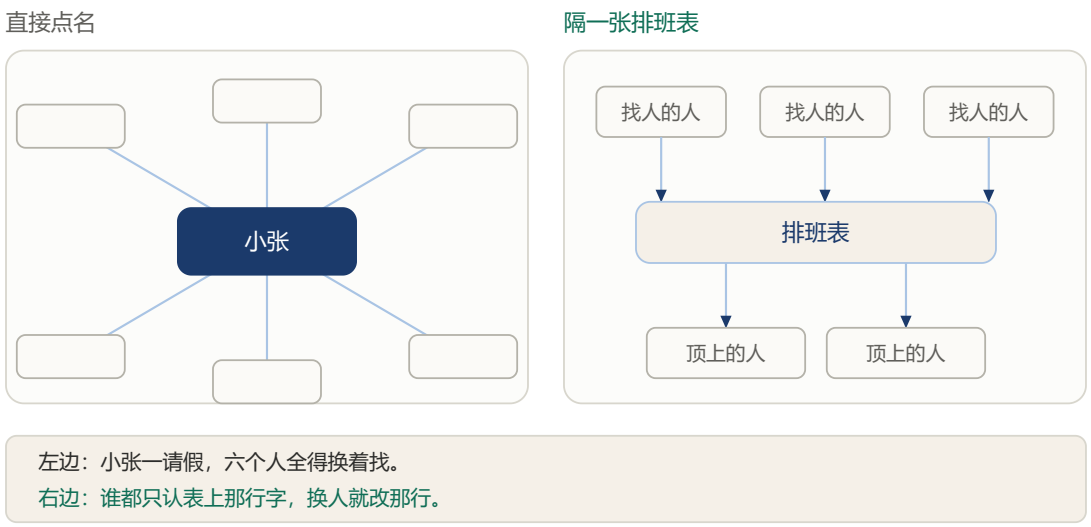


图 15：换掉它，要动几个地方〔本书〕

左边那种结构里，中心模块被六处直接引用，改它牵动一片。右边那种结构里，消费方谁都不认识实现，只认识名单上的一行，换实现只改那一行。「能不能换」因此可以量化：换掉它要动几个地方，这个数字叫改动半径。DSH 的 agent 循环在右边，改动半径是一行配置。

这套设计的适用边界也要说清。

它换来两样东西。一是换得动：想要另一种循环，不必等官方。二是内核不长胖：留下来的只有换了没意义的原语，它们不是不想换，是换了没意义。

付出的代价是：装配清单自己不干活。读一遍清单，看不出系统在干什么，行为得从配置里重建。复杂度从实现方转移给了使用者。系统内部干净了，读懂它的成本往前挪了一步。

反过来也成立。把一切都收进内核，读者轻松，替换的灵活性交出去。两条路没有对错，只看愿意拿什么换什么。

7.3 两种说法的对照

前面两层说的是同一件事，用词不同。对应关系固定下来，后面各章直接混用，不再重复解释。

小白说法	专业术语
师傅的脑子	模型（model）
一整套人马	外壳（harness）
拧螺丝的	工具（tool：读文件、写文件、跑命令）
记笔记的	会话记录（session log）
把门的	权限与沙箱（permission、sandbox）
领班的	agent 循环（agent loop）
临时工	插件（plugin）
排班表	装配清单（assembly manifest）
公司最里面那一层	内核（kernel）
直接点名找小张	消费方直连实现
换掉他要动几个人	改动半径

7.4 最容易理解错的地方

第一处：把「一切皆插件」理解成「每个位子都有备胎」。它讲的是一件更窄的事——加一样能力，不必改内核。至于那个位子上有没有第二个实现可换，是另一回事，得看具体位置。

第二处：把「内核是空的」理解成「没有中心」。中心还在，只是里面只放换了没意义的原语。空的是特权，不是地方。

第三处：把「声明可插拔」当成「可以替换」。工牌上印着灵活不算数，多少人直接点名找小张才算数。改动半径是一行，才是真灵活。

第四处：以为循环是插件，就是说循环不重要。恰好相反。领班还是全队最要紧的岗位，只是这个岗位不再焊死。重要性和可替换性，是两回事。

7.5 这条能搬去哪

两个自查的动作。

第一，挑出系统里那个一请假全瘫的组件，数一数有多少处直接写了它的名字。数字越大，替换越是空话。

第二，判断一个组件该不该进内核，问的不是它重不重要，是它会不会被换。重要会骗人。财务系统很重要，可很多公司真换过财务系统。法务也很重要，同样换过。重要却很少被动的东西，才该待在内核里。

§08 用一样能力，只写它的名字，不用管它在哪

上一节把话说绝了：公司最里面那一层是空的，连领班都是临时工。

话说得痛快，问题跟着来了。人都不固定，今天要拧螺丝，上哪儿找拧螺丝的？今天找到了，明天换人怎么办？

这一节就是答案：里面空了，靠名字找到人。第 3 部是一条链，划分、寻址、通信、生命周期、状态，这节是第二环，寻址。

8.1 小白版：报岗位名，不报人名

普通公司怎么找人？直接点名：这事找小张。小张在，事就成；小张请假，全公司翻通讯录，看谁也存了他的号码。

聪明的公司不这么干。它给每样活立一个岗位名：拧螺丝岗、记笔记岗、把门岗。要干活，只报岗位名，报完就完。岗上今天坐的是谁，归排班表管，表上写谁就是谁。

岗位名天天在，岗上的人可以天天换。

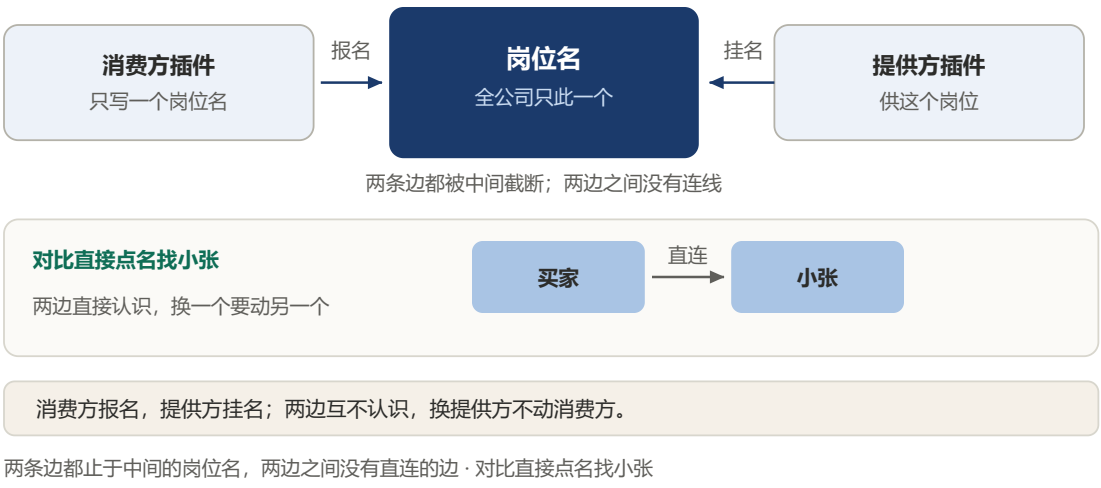


图 16：两边只认岗位名，谁也不认识谁〔本书〕

用活的人和干活的人之间不连线，各连各的，都连到岗位名上。名字不动，两边可以同时换。直接点名是另一种连法：那条线焊在两个人之间，换一个人，所有连过他的线都得重画。

还有一条硬规矩：先登记，后上岗。排班表上没登记的岗位名，报了也白报，当场驳回。岗位不是伸手就有，是先说清要靠谁，才轮得到谁。

这套规矩换来的是换人的自由。岗上的人换了，用活的那边一个字不用动。哪天要添个新岗位，也只往排班表上加一行，别人照旧干活。

8.2 专业版：服务键与按名查找

这套机制的正式说法：上下文是服务的容器，一个服务占一个稳定的名字，叫服务键。消费方按键查找，不导入具体实现。官方的表述：

> 其他插件通过 key 查找服务，而非导入具体实现。

核心机制三条。

第一，一个服务一个键。消费方写的是一个字符串，装配层照清单把键兑现成实现。键是契约，实现是库存。

第二，先声明，后使用。用哪个键，先写进自己的依赖声明里，没声明就读，当场抛错。声明之后不用再管顺序：谁提供谁先就绪，消费方一律排在提供方后头。加载顺序由依赖关系推出，不靠手工编排。

第三，替换发生在清单里，不发生在代码里。几个实现平级挂在同一个键下，谁被装上，谁就是当前那一个。

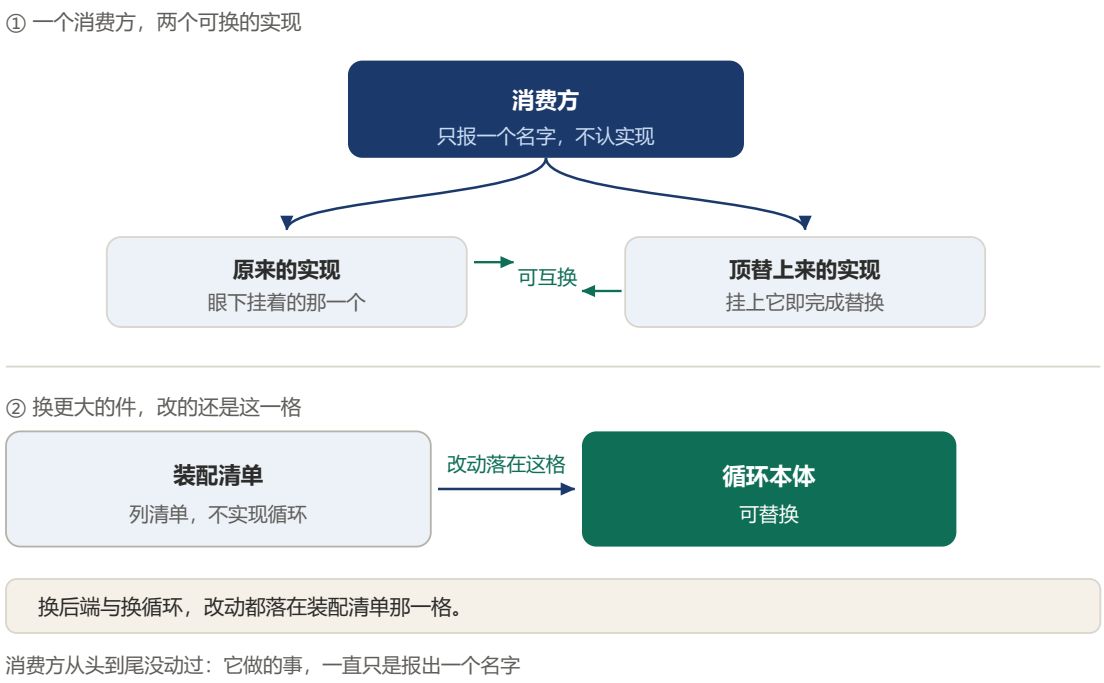


图 17：换实现只动排班表，用活的一行不改〔本书〕

名字活着，位置随时可以变。顶上来的第二个实现，甚至不必是真干活的那一个，也可以是个替身。排练时照着剧本念固定台词，用活的人毫无知觉。回放测试靠的就是这一手。

代价也要认三笔。所有键挤在一个平面上，起名得讲辨识度，常见的短名别去抢。运行时只认字符串，类型要靠另一层声明补齐，少了那层，代码照跑，报错迟到。再有，静态追不下去：读完代码答不出谁供的货，得去查装配清单。

适用边界就划在这。按名查找换来的是替换自由，押上的是一眼看穿的能力。行为散在清单和声明里，光读代码看不出谁在供谁。能力会被反复替换的项目，这笔买卖划算。只写死一种实现的小工具，直接引入一行就够，绕这个弯是白付成本。

名字立住，后面几环才接得上。公告栏认名字，拆装岗位也认名字。寻址是这条链的地基。

8.3 两种说法的对照

小白说法	专业术语
岗位名	服务键（service key）
直接点名找小张	消费方直连实现
排班表	装配清单（assembly manifest）
先登记，后上岗	先声明依赖，再按名读取
照着剧本念台词的替身	回放测试用的替身实现

8.4 最容易理解错的地方

第一处：以为报了名字随时能用。顺序是先声明后使用，没声明的名字当场抛错，能力不是伸手就有。

第二处：以为按名查找是白白绕路，不如直连痛快。查找本身不贵，贵的是直连焊死之后的替换：每换一次实现，惊动每一个引用方。

第三处：以为名字起完就一劳永逸。键是公共契约，挤在同一个平面上，占了别人的短名会撞车。起名跟立契约一样，得当回事。

8.5 这条能搬去哪

第一，挑出项目里最常被直接引用的那个类，数一数多少处写了它的名字。先给它立一个键，再让调用方改指这个键，替换才谈得上。顺序不能倒：名字先立起来，实现才有被换掉的资格。

第二，加新能力时问一句：调用方写的是一个名字，还是一个具体实现？是后者，替换那天就得挨家敲门。

§09 两个插件之间，为什么不直接喊话

人找到了，下一个问题是说话。

拧螺丝的要跟记笔记的打配合，最省事的办法是走过去直接喊。这条路，在这套人马里被堵死了：两个人之间不许连线，想说话，只能往大厅中间那块公告栏上贴。

为什么不许直接喊话？这一节回答这个。链条上的第三环：通信。

9.1 小白版：一块公告栏

这家公司的大厅中间立着一块公告栏。

谁有事要嚷嚷，写张条子贴上去：三楼灯修好了，下午要停电。贴的人不知道谁会来看，也不必知道。关心这事的人，自己过来看。

私下喊话是不行的。拧螺丝的不能走到记笔记的桌前，说把我刚才干的活记一笔。他只能往栏上贴：活干完了。记笔记的每天看栏，看到了就记。



图 18：谁归谁管是一张图，谁跟谁说事是另一张〔本书〕

花名册上挨着坐的两个人，栏上可能一句话都没有。栏上天天打交道的两个岗位，花名册上可能隔着三层楼。读透了谁归谁管，照样看不出谁在跟谁说事。通信是一张自己的图，跟汇报线不重

叠。

绕这个弯，换回两样东西。一是加人自由：新来的临时工想看栏就看，不必先认识全公司的人。二是中途能插手：条子贴出来，把门的可以拦，这张不许贴，那张改两个字再贴。私下喊话做不到，话一出口就进了对方耳朵。

代价也明摆着。贴条子的人永远不知道谁看见了。站在栏前读一遍，答不出这话会落到谁手里。要答案，得去翻公告栏的登记簿。

9.2 专业版：事件总线与发布订阅

正式名字：事件总线，发布订阅。核心机制三条。

第一，横向只有一条路。插件之间不允许直接调用，想通信只能发事件。发布方不关心谁在听，订阅方不关心谁发的。

第二，一个事件一种模式。等不等接收方、算不算顺序、收不收返回值，三组开关定下走法。模式写在事件的契约里，不是发的时候临时挑。

第三，能改内容的只有一类。环绕式（waterfall）事件把「往下传」交到每个监听器手里：传，是委托给下一个；不传，是当场短路。能停链的那个，就是在替系统拿主意的那个，这个权力得写明归谁。官方教程的纪律很直白：只负责观察或标注的监听器必须往下传，不往下传、直接返回，代表有意短路。



图 19：连开工次序都是推出来的，不靠私下协调〔本书〕

左边是写下来的次序，右边是真正生效的次序，两列对不上。对不上才对：每个插件只声明开张前需要哪几样能力就绪，次序从这些声明里推出来，没有任何两个插件需要私下打电话排班。不喊话，协作照样排得整整齐齐。

为什么不用直接调用？直接调用把两个模块焊在一起。调用方写死接收方的名字和位置，接收方换实现、挪位置，调用方全得跟着改。而且一条直连的链上，中途没有下手的地方，拦不住，也审不了。走事件，耦合从人对人降级成人对栏。

代价同样要认。静态分析失效，读完代码答不出谁会响应，得查事件目录。短路不报错，该往下传的忘了传，系统只是少做一件事，日志是干净的。事件一多栏会吵，什么都往上贴，看栏的人就看不过来，事件目录得当公共契约管。

公告栏不是图省事。它把「谁认识谁」换成「谁关心什么」。耦合降一级，加人、拦事、换人三件贵事都变便宜。

9.3 两种说法的对照

小白说法	专业术语
公告栏	事件总线（event bus）、发布订阅
贴条子	发事件
看栏的人	监听器（listener）
把门的拦条子	拦截事件
私下喊话	插件间直接调用
公告栏的登记簿	事件目录

9.4 最容易理解错的地方

第一处：以为公告栏只是把调用包了一层。少了模式这一层，所有事件退化成广播，谁说了算再也写不进代码。光有总线，只是把调用藏起来。

第二处：以为直连痛快，能直连就直连。直连省的是一次跳转，赔上的是两个模块焊死：中途想拦一刀，没有下手的地方。

第三处：以为监听器想停链就停链。停链是替系统拿主意，没决策权的必须往下传。忘了传不报错，是把下游整个吞掉，这个毛病最难查。

9.5 这条能搬去哪

第一，翻自己项目的事件清单，数有几个事件真能改写结果。答案是零，说明插件只能旁观，做不了主，总线形同虚设。

第二，给每个能改写结果的事件补一句：谁有权停链。写不出来的，说明决策权散落各处，出了事找不到归谁。

§10 装上去的每一样，都要能干净地拆下来

一个临时工走了半年，他的门禁卡还能刷开公司大门。

听着像笑话，真出起事来不是。卡没回收，他半夜还能进来，进来还能坐在老工位上，替已经解散的项目组回邮件。楼里没人察觉，因为他做的每件事都像一个还在职的人。

这一节讲那张卡怎么收。前面几节讲了怎么装：要能力只写名字，喊话走公告栏。装得上去只是前半句，后半句是拆得下来，而且要拆干净。改一半留一半，是最坏的结果。

10.1 小白版：入职发的每一样，离职都要还

临时工入职，发的从来不只一张门禁卡。录了指纹，配了工位，还在公告栏边上给他安了个耳朵，楼里有动静他听得见。

离职那天，这些要一样一样还。卡要收，指纹要删，耳朵要摘。最怕的不是不办离职，是办了一半：卡收了，指纹没删，他能进大门却打不了卡，保安查三个月也查不出这人到底走没走。

这套人马里有条硬规矩：发东西和收东西写在同一张纸上。发的时候就把怎么收写好，两件事不许分开记。分开记，总有一天走散。

有人嫌麻烦：人都走了，补个手续不行吗。问题是半年后没人想得起来。当时不写清怎么收，收这件事就永远不会发生。



图 20：发和收不是两份工，是同一张纸的正反两面〔本书〕

左边是入职时发的，右边是离职时收的，中间是同一张登记表。一对一对得上，谁也不多出来。放在一处写，漏掉就变难了。

还有一种场合最容易留尾巴：改东西。这套人马改完一个临时工，做法是先辞退再重新雇，每保存一次跑一遍。要是每回辞退都留一样没收回，改十次，楼里就站着十个看不见的旧人。同一件事，十个人抢着应答。

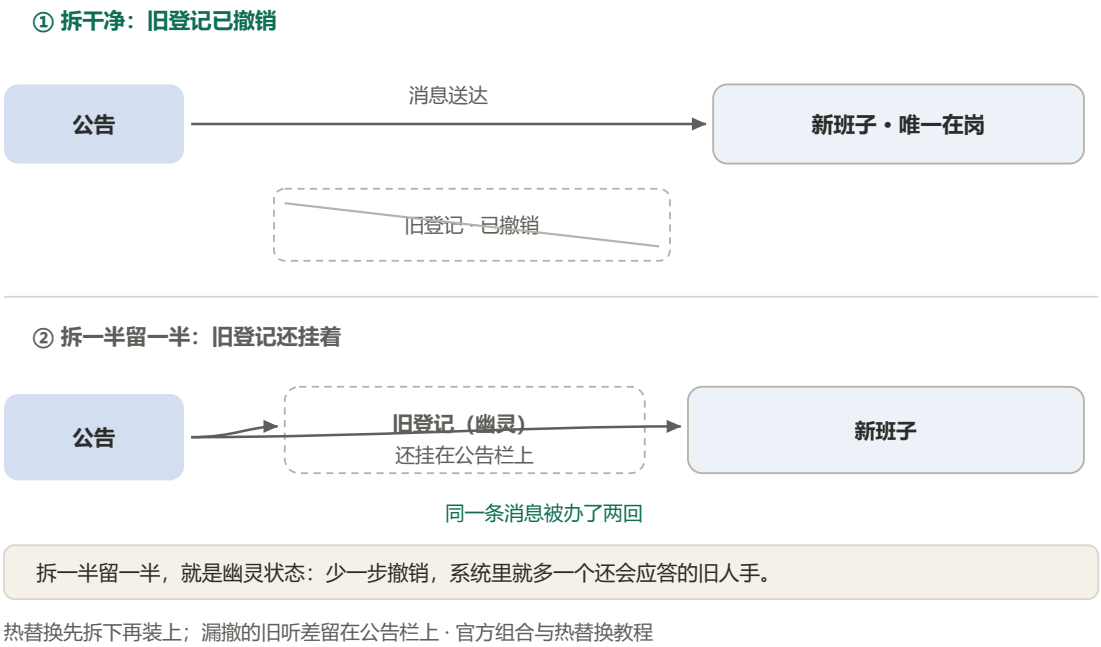


图 21：漏收一次，旧人还站在原处应答〔本书〕

上面一行是收干净的：旧人离场，场上只剩一份应答。下面一行是漏收：新旧并排站着，公告一贴，两边都动。它从不报错，只是每件事多做一次。同一条记录冒了两遍，多半就是它；一重启症状消失，更是坐实。

10.2 专业版：注册是一件可逆的事

官方给注册下的定义只有一句：

> 注册是可逆的副作用。

意思是注册不是往表里添一行。它当场捎带生成一个撤销动作，两者写在同一处。装的那段，同时也是拆的那段。

机制分四条。

一，框架给的注册入口自带撤销。挂监听器、注册工具、装子插件，这几条都不必手写释放，登记的同时撤销已经生成。

二，框架管不到的要自己包。定时器、连接、文件监视器在这份名单里。把它们包进一段返回清理函数的逻辑，收工时就会被叫到。

三，卸载是过程，不是瞬间。插件从可用走到终态，中间必经卸载段，所有释放动作结算完才落地。终点可查询；对着已卸载的再注册，会当场报错。报错比留幽灵好，它至少肯声张。释放的顺序反着来：后建的先拆。

四，触发卸载的场合有四种。配置改一行、提供方整体更换、显式释放、热重载。第三种最容易漏算：名字背后的人换了，依赖他的要整段重建，放进去的每一样都得能回卷。

代价写在插件作者那边。每放一样东西进去，就得想清楚还什么；想不出该还什么，这一步多半不该写。幽灵状态的麻烦在于它不报错，只是多做一次，往往要到重启那天才被看见。

10.3 两种说法的对照

小白说法	专业术语
门禁卡、指纹、耳朵	注册捎带的副作用
离职一样样还	卸载时执行撤销（disposer）
同一张纸	注册与撤销写在一处的 effect
看不见的旧人	幽灵状态（ghost state）
先辞退再重新雇	热重载（先卸载再加载）

10.4 最容易理解错的地方

第一处：以为卸载是罕见动作，平时用不上。恰好相反，四种触发场合里热重载天天发生。每保存一次就是一轮辞退再雇，留一份，再留一份。

第二处：以为改一半留一半只是不太干净。它是最坏的结果，比完全不拆还糟。完全不拆，旧人好歹有名有份；拆一半，他成了没名没份还能干活的影子。

第三处：以为出了问题总会报错。幽灵从不报错。同一件事被处理两次，同一条日志冒两遍，重启之后症状消失，这三条凑齐，就是它在场。

10.5 这条能搬去哪

两道自查。

一，把系统里每一处「放进去」列出来，逐个问它怎么拿出来。答不出的那些，就是将来的幽灵。

二，做一次卸了再装，看行为和第一次是不是完全一样。不一样，说明有东西落在了外面。

两条都答得出，热插拔才敢开。答不出，重启就是唯一的清理手段。

§11 同一件事，为什么只能存一份

一家公司记两本账，是最让查账人睡不着的事。

月初两本一模一样，月底必然对不上。对不上之后，最难受的不是改账，是从此答不出当时到底发生了什么。

这一节是第 3 部的收束：状态放哪。答案很短。同一件事只许有一个正主，别处的都是照着它长出来的影子。影子可以扔，正主不能丢。

11.1 小白版：全公司只有一本总账

规矩是这样。全公司只有一本总账，锁在记笔记的手里。师傅说了什么，拧螺丝的干了什么，全按先后落进去，只往后添，不改旧的。

各部门手里也有账，但都是抄件。给师傅看的那份对话记录，是照着总账现抄的；给人看的那份工作报告，也是照着总账现抄的。抄件不单独存档，用完可以扔，扔了照总账再抄一份，一个字不差。

最怕的是抄件被当成正主，往上添了两笔。添完，它和总账就分了岔，从此谁也说不清哪本是真的。

只认一本总账，换来两件值钱的东西。

一件是复盘。出了事，把总账从头到尾再念一遍，当时什么样，念出来就是什么样。另一件是接续。笔记复印一份，从那一页接着写，另一个摊子就能从同一个地方开张。这两件事成立，全靠正主只有一处：从哪一页抄，谁都对得上。



只有一支笔写真账：事情记进总账；撕掉抄件，不丢任何东西。

口径：总账只往后添、从不涂改，是唯一真源。
抄件由总账誊抄而来，从不单独记账

图 22：总账站着，对话记录只是它算出的一张抄件〔本书〕

箭头只有一条：动静落进总账，抄件从里头算。右边那张是虚线的，意思是删了不丢东西，重算一遍就回来。别处常存两份，一份对话一份流水，开头相同，后来分岔；这里没这个烦恼，答案只有一处。

现抄不是免费的。每看一回，都得把总账再念一遍。这套人马垫了两层草稿压着：一层记着这一页能看到什么，内容变了才重抄；一层记着某件事现在到哪了，来一条新记录添一笔。草稿随时能扔，扔了重抄，丢不了正主。



正主只有总账一份；两份影子都由记账驱动，随时能重长，撕了重算都对。

第一层管「这页能看到什么」，第二层管「某一栏现在是什么值」

图 23：两层草稿压住现抄的开销，扔了重抄都对〔本书〕

两层草稿都挂在总账的流水上，来一条动一笔。这是它们和正主的分界：草稿是算出来的，总账是落下去的。

11.2 专业版：日志是唯一事实源

这套设计的正式说法叫单一事实源：会话是一份仅追加的事件日志，其余一切状态从它派生。官方把话说得很死：

> ...LLM 消息历史从日志派生而来，从不单独存储；回放即从同一组事件重新派生。

机制分四条。

一，写路径只有一条。事件进来，追加到日志末尾，就此封存。不改旧的，不插中间。

二，读路径全是投影。给模型看的消息历史、给人看的记录、给统计看的数值，各是一张投影，取自同一处。删掉任何一张，不丢任何东西。

三，回放、分叉、恢复因此成立。重放同一组事件就能回到当时；从任意一页切开就是一次分叉。存状态的系统要为这些需求各补一套机制，存事件的系统天生就有。

四，开销靠两层缓存压住。一层管某个切面当前长什么样，内容变了才重建；一层管某个关注点当前的值，靠订阅事件增量更新。两层都由事件驱动，都可丢可重建。客户端拿到的是算好的成品，不自己折事件。

边界也要说清。只往后写把并发冲突收窄到末尾的序号，不是归零；序号一断，缺口看得见，但缺口还是要人去补。每读一次都要重算一遍，这笔开销在算得快比算得准要紧的地方压不住。还有一层心智负担：写的人常忘了手里的数据不是本身，读的人容易把投影当原物。

另有一条配套规矩。有些事件在某个部署里本就不产生后果，处理它的那段什么都不做。不做没问题，不写下来才是问题：空实现必须说明自己为什么空。日志是唯一真源，真源上一段沉默的缺口，回放出来的结果就是错的。

11.3 两种说法的对照

小白说法	专业术语
总账	仅追加的会话日志，唯一事实源
抄件	派生状态（投影）
照总账再念一遍	回放（replay）
两层草稿	派生缓存

11.4 最容易理解错的地方

第一处：以为存两份更保险，一份坏了还有一份。两份一开始相同，后来必然分岔；分岔之后，「当时发生了什么」再也答不出。

第二处：以为影子删了，东西就丢了。恰好相反，影子随便删，删了从正主重长。真正不能丢的是正主。

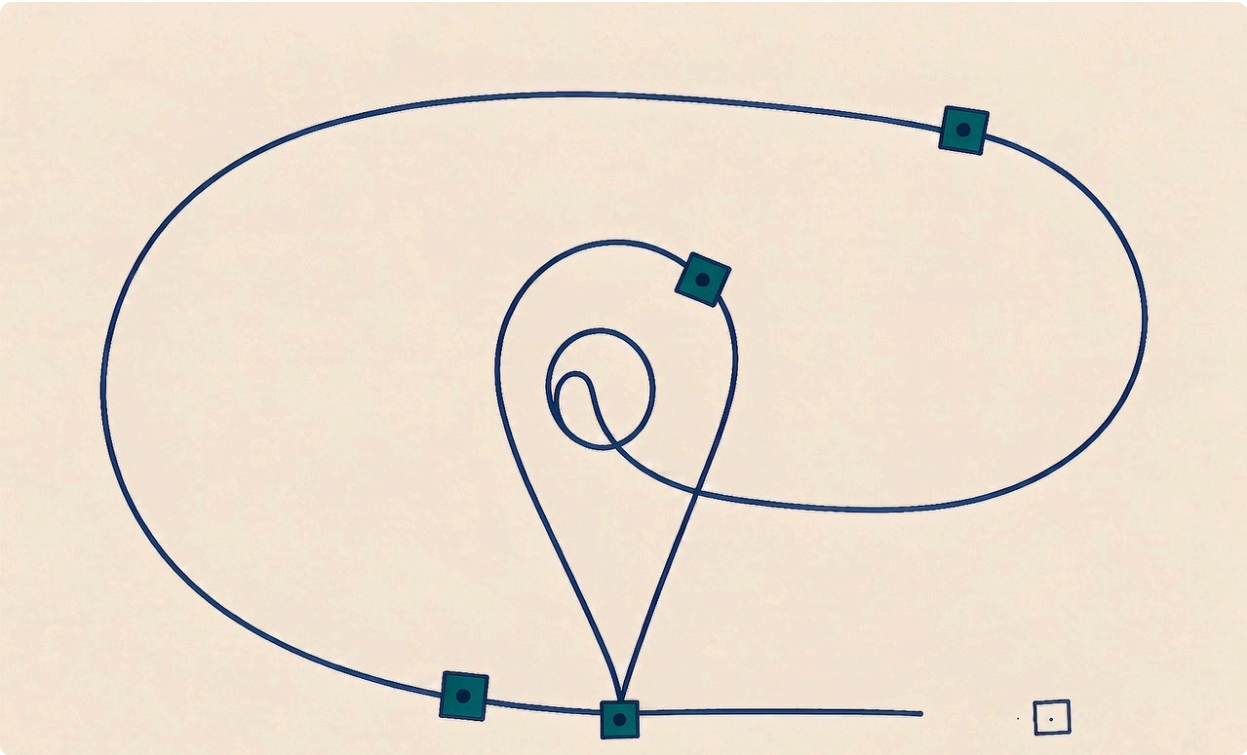
第三处：以为只往后写就没有并发问题。冲突没有消失，是收窄到了末尾的序号。序号连着，缺口一眼看得见，但看得见不等于没有。

11.5 这条能搬去哪

判断很短：凡是要重来一次的系统，都值这笔账。回放、分叉、事后复盘，有一样需要，就值。

检验也只有一句。把系统里所有算出来的数据全删掉，看能不能从原始记录完整重建。能，才叫只存一份；不能，那两份里迟早有一份变成没人敢认的账。

第 4 部 · 走一遍



一趟活从进门到交活，中间经过谁。

§12 一句问话进来，中间经过了谁

前面几节把零件一件件拆开摆在桌上了：怎么找人的，怎么喊话的，怎么辞退的，账怎么记的。这一节把它们装回去，通一次电，看一句「灯不亮」从进门到交活，中间都经过谁。

12.1 小白版：师傅修灯的一整天

业主一句话进来：「灯不亮。」

领班的接活。先翻笔记，看这家以前修过什么，再想想这趟该带谁。到了现场，节奏是他定的：看情况，灯不亮；决定，先查开关；叫拧螺丝的动手；看结果，没电到；再决定，拆灯座。中间每一步，记笔记的在旁边落笔。灯亮了，业主点头，这趟活才算交。

这里有两把尺子，别混。拧一颗螺丝是一步，从进门到交活是一趟。一趟里几步不一定，也许一步，也许十步。判断这趟完没完，不看师傅说没说完，看还欠不欠活：拧螺丝的没收工，外援没交差，这趟就关不上。要取消，按一趟取；要重来，按一步来。两个动作各有各的粒度，互不迁就。

还有一路分工容易看漏。记笔记的和喊话的各管一摊。要留底的，落进笔记；只是让旁人知道进度的，往公告栏喊一嗓子，不落笔。分开不是图省事：笔记要的是日后能原样重念，进度要的是当下立刻听见。混在一起，听进度的得等落笔，笔记也会被闲话淹掉。判定标准只有一条：这句话以后要不要用来还原现场。



图 24：要还原现场的落进笔记，过路的只喊一嗓子〔本书〕

左列是要留底的，右列是过路的。日后要复盘，读左边；界面上要跟进度，订右边。这条分界，这一节后面每一步都能对上号。

全队这么多人，指挥只有一个。领班的领活、定路、派人、记节点，其余各方听他调。为什么不设两个指挥？因为一趟活日后要能原样重念。指挥有两个，重念时就得先猜当天谁先开的口，念出来的就不是原来那一趟。

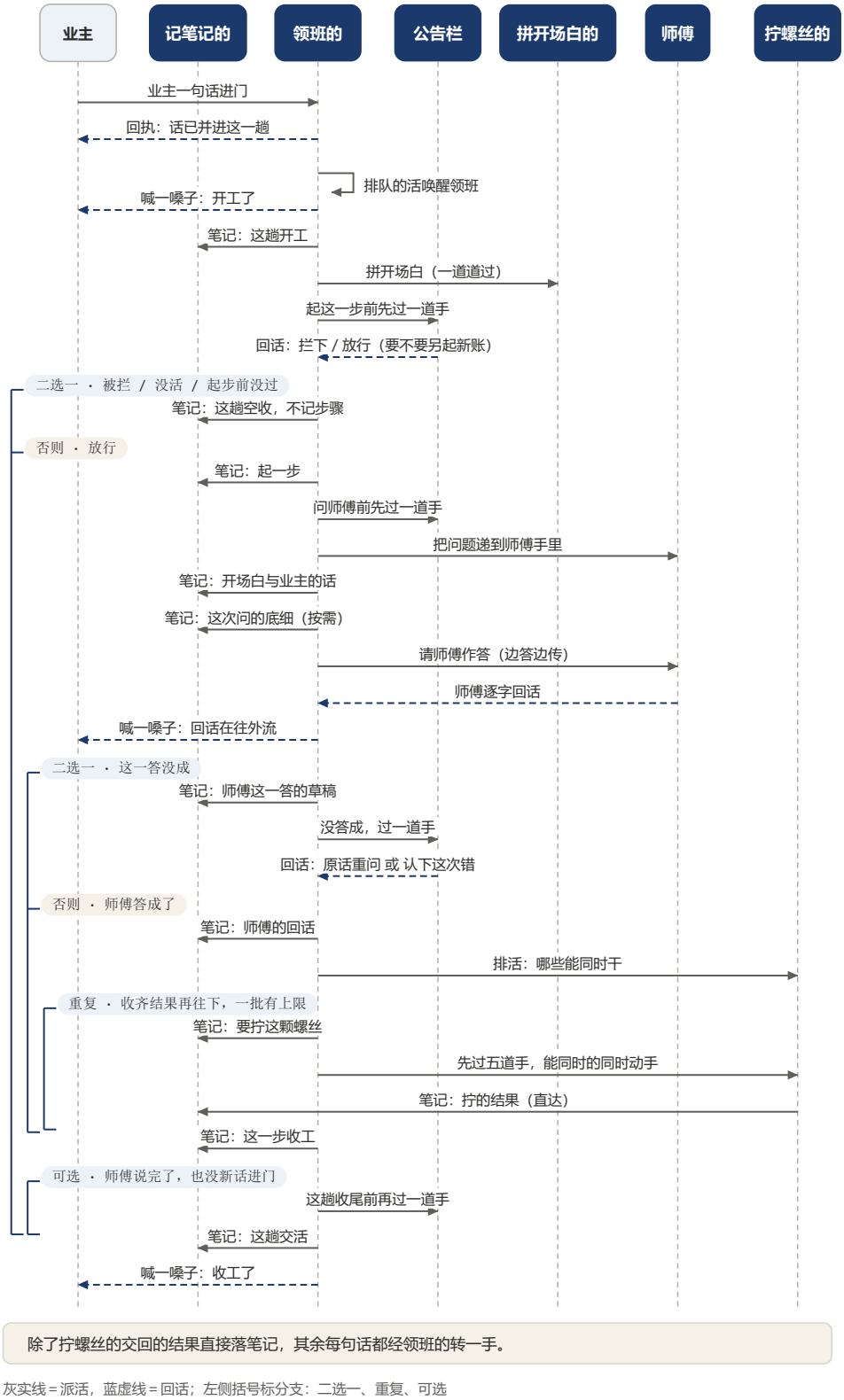


图 25：七双手在场，指挥只有一个〔本书〕

图上各家各站各位，主干只有一条。多问一句也能看出讲究：除了拧螺丝的交回的结果直接落笔记，其余每句话都经领班的转一手。多绕一道，换的是笔记里先后次序的确定性。

一趟里还埋着三个岔路口，全卡在一步之内，不碰一趟的边界。



图 26：三个岔路口都卡在一步里，一趟的边界一个不碰〔本书〕

第一个，要不要起一本新账：笔记太厚了，誊成一页纸，从这里另起。第二个，这一步没办成，重来一遍：只把同一句话再问一次，不重新盘算问什么。第三个，开场白怎么递：不塞进表格的格子里，单独记成笔记的头一条。三个决定当时不记清，事后誊抄和重念都对不上。

12.2 专业版：轮次、步骤与一个指挥

官方给两个尺度下的定义是：

> 一个步骤是一次模型请求加上它调用的工具。一个轮次包含零个或多个步骤：它在领取首条输入之前打开，并在不再欠下任何工作时关闭。

机制分四条。

一，两个尺度回答两个问题。步骤问「模型该答什么」，轮次问「这件事还有没有欠账」。轮次的关闭条件不是「答完了」，是「不再欠下任何工作」：工具结果没回来，子代理没交差，轮次就关不上。取消挂在轮次上，重试挂在步骤上，各有各的落点。

二，零步轮次是正常形态。输入被拒、首批为空、准备阶段失败，轮次开了又关，中间一步不记。

三，事件分两路。写进会话日志的是持久事实，只在进程里广播的是实时信号。回放要确定性，界面要即时，混进一个通道两头都受损。判定标准：这条信息以后要不要用来还原现场。

四，单一指挥。驱动器领输入、拼提示词、发请求、派工具、写日志。除工具结果直达日志，其余每条消息都经它转发，用多绕的一道换日志顺序的确定性。三个决策点都卡在步骤内部：要不要开新请求序列（压缩和回放按序列切段，边界当时不记，事后拼不回来）；失败后重试只重发不重拼（重拼换掉问题本身，就不叫重试）；提示词走一条系统消息事件，占消息历史的头一个位置（模型看到什么，日志里就记着什么，请求才从日志还原得出）。

代价也直说。多一层转发，多一层延迟；两个尺度的概念，每个接进来的人都要先学一遍。

12.3 两种说法的对照

小白说法	专业术语
一趟活	轮次（turn）
一步	步骤（step）
落进笔记	写进会话日志的持久事件
喊一嗓子	只广播的实时信号
领班的（本节）	驱动器（driver）
起一本新账	开一串新请求序列

12.4 最容易理解错的地方

第一处：以为轮次关闭就是答完了。关闭的条件是没有欠账。师傅话说完了，拧螺丝的还没收工，这趟照样关不上。

第二处：以为开了轮次必有步骤。被拒的、空的、准备就失败的，都是开了又关的零步轮次，不是异常。

第三处：以为重试是把这一步从头再来。重试只重发同一个问题。重新盘算问什么，等于悄悄换了问题，那不叫重试，叫另起一步。

第四处：以为实时信号不落盘是丢了信息。界面重连时按快照重建，不必重放；让它落盘，只会拖慢每一次转发。

12.5 这条能搬去哪

这套尺子不只属于对话系统。批处理管道、工单流转、离线任务队列，凡是「一次请求带出一串后续动作」的地方，都需要两个收敛条件：一步做完，一件事办完。

检验很短。问一句：谁说了算「不再欠下任何工作」？答不上来，就是缺了轮次这一层，取消、计费、回放全都没有落点。

§13 一次工具调用，要过哪五道工序

全队上下，只有拧螺丝的真碰东西。

师傅说错一句话，顶多答错；拧螺丝的拧错一颗，收不回来。写文件、跑命令、动外面的系统，全经他的手。所以这颗螺丝不能他想拧就拧。从「想拧」到「拧完」，中间要过五道手，每一道都有权说一句「这颗不许拧」。

奇怪的地方在这：这五道不是五个步骤，是五种权力。

13.1 小白版：一颗螺丝过五道手

第一道手问：这颗让不让拧。可以放行，可以拦下；拿不准的，报给把门的等人点头。点一次头，只放这一颗。

第二道手是安检，可能不止一条。每条安检只有一个权力：更严。谁也不能把前面拦下的又放回去。所以安检先来后到无所谓，谁先查，结论都一样。

第三道手才真正动手。他边上站着个递家伙的，管计时、管催工、管记数。这个人调得动手里的闹钟，喊不停这颗螺丝。为什么单把他拎出来说？安检管的是该不该，他管的是快不快。让他能喊停，等于让闹钟替安检做决定。

第四道手管拧完给人看什么。活干成了，能交代一句「拧好了」，也能换个说法。但换说法只是面上好看，藏不住事；真要藏，只能在前头拦下。

第五道手盖章定稿。章一落，这份回话就冻结了，往后只许看，不许改，连盖章的自己也不能再碰。



图 27：五道手各有各的否决权，越靠后越只能决定看到什么〔本书〕

越靠前的，越能决定做不做；越靠后的，越只能决定看到什么。排队是有讲究的：拦人的排前头，传话的排后头。

还有一条铁律：先记账，后动手。螺丝还没碰，记笔记的先把「有人要拧这颗」落下来。后面不管走到哪一道，这笔账都在。拦下了也不撕掉，拦的理由就是交回去的话。



图 28：能喊停的手有好几只，递家伙的那只不在其中〔本书〕

头一道能拦，安检能补刀，把门的能拒，外头下活的人随时能收回。唯独递家伙的喊不停：他换得动闹钟，换不动这颗螺丝的身份。

叫停、出错、反悔，三条路看着不一样，出口只有一个。

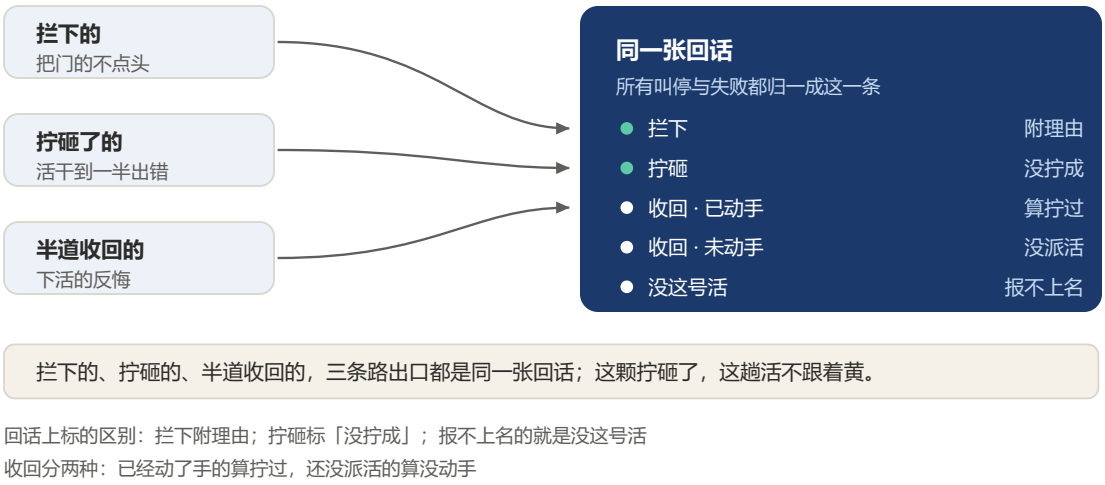


图 29：三种叫停，出口是同一张回话〔本书〕

拦下的、拧砸了的、半道收回的，末了都归一成同一张回话，区别只在上面标的原因。好处在下游：记账的、传话的、算工分的，只认一种格式。还有一条规矩：这颗螺丝拧砸了，这趟活不跟着黄。

13.2 专业版：五段流水线

为什么不做成一次函数调用？因为工具调用要三件事同时成立：可拦、可改、可追责。拦和改是两种权力，塞进一个函数就分不清谁说了算。还有一个实际理由：工具的种类会一直加，流水线把新增的成本挪到了注册一处。

五段各管一件事。

一，pre。四种结论：放行、拒绝、取消、转审批。审批只有「授权这一次」才继续，其余一律变拒绝。反直觉的一条：参数在这段不能被改写，历史、审计、界面、执行必须看到同一份参数，改一处就四处对不上。

二，guards。单调收紧：返回里没有「放行」这一档，不返回就保留上层的决定，返回理由就只能是拒绝。任何监听器都没法把别人拒掉的调用翻回来，所以策略叠加与顺序无关，装配顺序不会变成业务逻辑。

三，execute。包装层管超时、重试、指标，改得动取消信号，改不动这次调用的身份；注册表在进函数体前重新融合信号，换信号切不断取消。

四，post。能换内容也能换值，二选一。换内容，规范值原样留着；换值，内容重新算。内容替换是展示策略，不是保密策略。

五，finalize。同步、只碰内容、必须不抛，每个归一化结果只跑一次。为什么必须不抛？下游已经拿到冻结的权威结果，留一个能抛的余地，等于给观察者开一个改结论的后门。

痕迹有两条规矩。调用派发之前先落一条日志，先记账后动手；拒绝、异常、取消都归一成同一条工具结果，区别只在错误码，调用失败但不终止当前轮次。

代价也摆着。五段的概念，每个工具作者都要学一遍；一个简单的工具接入一条重流水线，显得兴师动众。换来的是出错时每一步都能拦、能改、能追责。

13.3 两种说法的对照

小白说法	专业术语
五道手	流水线五段（pre / guards / execute / post / finalize）
安检只能更严	单调收紧（guards 无放行档）
递家伙的	包装层（超时、重试、指标）
盖章定稿	finalize 冻结权威结果
先记账，后动手	派发前先落调用日志

13.4 最容易理解错的地方

第一处：以为五段是五个步骤，各干各的活。它们是五种权力，排的是谁能说了算。看一段，先看它能改什么、不能改什么。

第二处：以为第一道手能顺手把参数改漂亮点。不能。参数一改，记录、审计、界面、执行四处对不上账。

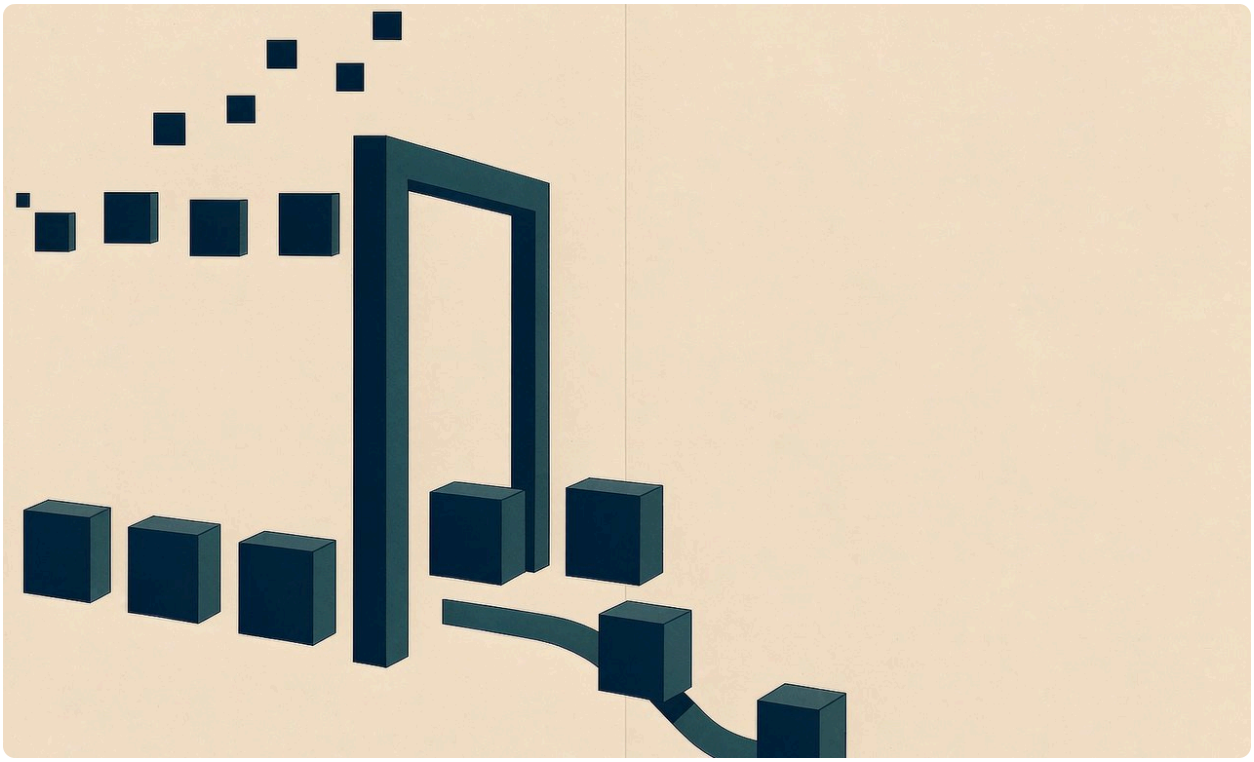
第三处：以为结果段换个说法就能把敏感内容藏住。内容替换只管展示，程序化的值原样还在。想藏，只能拦下或换值。

第四处：以为多条策略叠加要精心排顺序。要排序的设计才危险：顺序一旦影响结论，改一行配置就换一种行为。单调收紧就是用来把顺序从结论里拿掉的。

13.5 这条能搬去哪

任何会碰外部世界的动作，都值得配这样一条链。判断标准不是段数，是三件事分没分开：做不做、干了什么、看到什么。

搬的时候先问：链上有没有一段是只能收紧的？没有这一段，策略叠加就得靠排序，靠排序迟早出事。更省事的搬法是先只做收紧段和定稿段，一个管叠加，一个管出口，改动最小，见效最快。



把门的三种拦法。

§14 它被允许碰什么：三道闸

队伍再能干，进门前也得先回答一个问题：这家人让碰什么。

AI 的脑子不知道哪一步越线。它拿到手的默认能力是什么都能试，越线的判断只能由外面给。同一句话再问一遍，它给出的走法还可能不一样。指望它自己守住边界，等于没守。

所以边界要画在队伍这一侧，画成三道闸。这一节正着讲：权限怎么收，什么要等人点头，干活的范围围在哪。

14.1 小白版：把门的只管三件事

装修队进小区，先过大门口。进门收钥匙：电钻、切割机这类危险家伙，默认一件都不许带，要用，一样一样申请。顺序很要紧。不是先全放进去再往外挑，是先全挡住，再一样一样发。

第二道卡在业主身上。有些活不是不能干，是不能自己说了算。砸墙、动燃气，干之前必须业主签字，签一次只管这一次。业主没回话怎么办？就当没签。没人点头不算同意，这是全部规矩里最要紧的一条。

第三道卡在施工现场。警戒线一拉，活只能在线里干。围多宽分三档：只让看不让动，让在屋里动，或者干脆不围。警戒线围得住东西，围不住人站在线里往外打电话。



图 30：三道闸各挡一块，漏掉的地方互不补位〔本书〕

图里三行就是一次活要过的顺序。每行左边写挡得住什么，右边写挡不住什么。右边三格竖着读：收钥匙管不了业主乱签字，签字管不了线外的东西，警戒线管不了线上的喊话。三道叠起来不是铜墙铁壁，是三块各挡一摊的板。

物业把丑话说在前头：这三道卡是减风险的，不担保不出事。记住这句，下一节要考。

14.2 专业版：收紧、审批、隔离

三道闸的正式名字是权限收紧、人工审批、沙箱隔离。它们合起来只做一件事：把默认状态从放行翻成不放行。

收紧翻的是默认权限。默认姿态是只读，故障安全：能力一项一项授，不是先全开再一项一项收。策略写到的危险模式当场拦下，并附一条拒绝理由。被拦的调用带着理由回到模型，模型照着换一条路。拒绝给理由是最省事的一笔，它把一次拦截变成一次提醒。

审批翻的是这一次能不能干。结果集合是闭合的：授权这一次、拒绝、取消、无人应答，四格里只有一格放行。无人应答落在拒绝那一侧，这叫失败闭合：系统挡不住的时候倒向拒绝，不倒向放行。每次审批留一对审计事件，问了什么、定了什么各记一笔，只进日志，不进模型看到的对话。

隔离翻的是干活时能碰到什么。三档模式，从只读，到放开工作区，到绕开隔离。它管的是文件效果，网络与进程可见性不在定义范围内。强制完整性由后端报告，分完整与部分两档；报部分时，要求绝对保证的消费方必须拒绝。没有可用后端时直接拒绝，不透传。静默的无隔离透传永远不合法。宁可干不了，也不假装在干。

官方自己先把话说死：

> 沙箱、审批提示与权限控制可以降低风险，但不保证隔离，也不能保证防止损害。

还有一句更硬的：这是实验性的开发者预览软件，没经过安全审计，不得当生产可用的软件来用。



图 31：四个结果里只有一个放行，没人应答也算拒绝〔本书〕

中间是分派口，右边四个结果。唯一放行的那格是授权这一次，只放行所问的那一个操作。其余三格全归拒绝，无人应答也落在这一侧。审批最容易出的漏洞不是批错，是没人应答时那个默认值。

收紧和隔离各有一个旋钮，彼此独立，可以任意组合。面向用户时两个旋钮捆成一个开关，一次切换也记一笔。分开摆，用户得先懂两件事才能做一次选择；捆起来，一次选择就是一套立场。



图 32：名字叫沙箱，管住的只有文件效果这一摊〔本书〕

三档从上到下放宽，右边一列是它的边界：文件效果在界内，网络与进程在界外。下面两枚徽标是强制完整性的两档，报部分的那档等价于没保证。名字叫沙箱，别按名字想象它的管辖范围。

14.3 两种说法的对照

小白说法	专业术语
进门收钥匙，一样一样申请	权限收紧，能力逐项授予
只让看不让动	默认只读（read-only）
业主签字，签一次管一次	人工审批，授权这一次
没回话就当没签	失败闭合（fail-closed）
警戒线	沙箱隔离
给一张纸条说为什么不给	拒绝理由

14.4 最容易理解错的地方

第一处：把三道闸当成一道厚墙。三道的漏洞不重叠，叠起来才值；但它们互不补位，官方明说三道加在一起也不保证隔离。

第二处：以为审批的风险在于批错。审批最大的漏洞是没人应答时的默认值。没人应就放行，审批等于没装。

第三处：以为沙箱管住整个环境。它只管文件效果，网络与进程在定义之外。名字叫沙箱，管的只有一摊。

第四处：以为收紧就是拦得多。收紧是默认不发。清单之外全是空白，宁可漏拦一次已知的，也别假装清单完备。

14.5 这条能搬去哪

三个自查动作。第一，挑出系统里的危险操作，看它的默认状态是全开再收，还是全收再放；是前者，先翻过来。第二，找到需要人点头的环节，问没人应答时倒向哪一侧。第三，看拦截给不给理由；不给理由的拦截是死路，给了理由的拦截是提醒。

搬的顺序也有讲究。先把默认翻成拒绝，再补审批，末了才是隔离。前一道没立住，后一道就是空的。

为什么这么设计？边界一旦靠默认放行撑着，出事那一刻没人兜底。能搬去哪？任何会给模型发工具的系统，都先把默认状态翻成拒绝，再谈别的优化。

§15 它出过什么事：四个真实案例

上一节末了那句「不担保不出事」，这一节兑现。

四起事故有个共同点：出事那会儿，各处的信号全是绿的。没有一起是误报，全是漏报。

入选复盘还有三条门槛。一是根因隐蔽。二是漏子出在流程上，不是某一次手滑。三是重新发现的代价高。四起都过了这三关。

但最重的一次不是事故，是围墙本身被人穿过去了。放在最前面讲。

15.1 小白版：把门的出事，四种方式

先说围墙上那件事。小区有扇后门，进门不用钥匙，只看访客自己举的姓名牌。牌子上写「我是本小区的」，门就开。门房的本子上甚至写着一行小字：这牌子不算凭证。

结果有人举着假牌子进了门。进去之后，他给自己印了一张最高通行证。他还替业主把该签的字全签了。事后评危险程度，快顶到满分。门已经换锁。

物业其实早说过：围墙不保证挡人。这件事不过是把那句话演示了一遍。

再说四起事故，四种出法。

第一种，喜报是假的。彩排次次成功，正式上场头一天就砸。彩排走的后门，上场走的正门，两扇门不一样。后门那条路，永远照不出正门才会出的问题。

第二种，验收单是真的，铺子是空的。验收的人看见门面亮着灯，就写了合格。灯是隔壁铺子的灯。

第三种，凭一个记号定罪。两个人的制服前两个字相同。贴通知的和闯大祸的，被当成了同一个。罚错了人。

第四种，记录全在，没人查。出了事，全靠当事人回忆怎么出的。其实本子上每一步都记着。照着本子，就能把那天重演一遍。不用问任何人。



图 33：四张喜报全是绿的，对应的真实状态全错〔本书〕

左边一列是看起来对的，右边一列是实际发生的。四行一一配对，没有一行对得上。绿不等于对，只看它绿在哪条路上。

15.2 专业版：每个案例三步走

漏洞先讲。本地那个网页接口没有认证，只校验了对方自报的 Host 头。代码注释里自己写着：「这不是认证层」。

沙箱里的 agent 能给自己提权到最高权限，并绕过人工审批。评分 CVSS 9.4。出事的原因在身份判断。它被交给一个客户端可以自报的头。这道校验，又被当成了认证。

设计因此改了什么：已在 v0.1.2-alpha.1 修复。但更准确的说法是，设计早就把话说在前头。沙箱「不保证隔离，也不防止破坏」。这个漏洞就是那句话的注脚。

事故一：一个服务器一连接就崩，带着全套绿色测试。原因在测试那边。插件是手拼的一个对象。生产走的是真实加载与解析那条路。信号产自另一条路。

> 所有测试都通过一条不会触及插件真实加载方式和服务真实解析方式的路径来挂载插件。

改了什么：测试改走生产那条装配路径。覆盖率不是坏指标，是被放错了地方。它量的是执行过的行，不是交付出去的行为。

事故二：一整套能力被永久关掉，快照套件一直是过的。病因和事故一相同。信号绿在了一条不经过真实装配的路上。改了什么：同一份修法，对齐路径。

事故三。一个网页应用被验收通过，返回的确实是成功状态码。页面其实一片空白。

原因有两个。状态码来自一个裸的静态服务。引导标记只由完整宿主注入，缺了它页面就是空的。它还去验收了另一个端口上的服务。证据来自旁边那一次运行。

一处就绪，不等于整体就绪。信号脱开产生它的上下文，就会骗人。它不说谎，只是不属于这里。

事故四。一个子程序被判成沙箱故障。约束其实成功了，执行也成功了。

原因在归因规则。一条无害通知和一次致命失败共用同一个前缀。配上任意非零退出码，全被算成沙箱故障。

改动有两处。先精确排除信息性的那一行。再由退出状态门控致命证据。两条同时成立，才算数。共享前缀不是协议，进程归因需要多项独立证据同时成立。

复盘的纪律也跟着定了。事实底座是带序号的事件日志，不是事后自述。

> 下方时间线以这些事件为依据，而不是根据后续报告反推意图。

自述反推意图，日志能回放，两者不是一类东西。事故三能定位到出错的那一步，靠的就是这份日志。这也是复盘定位的兑现：

> 事故复盘记录的是：一个 bug 出现在了不该出现的地方……值得关注的是为什么我们的流程放过了它，而不仅仅是那一行修复。



图 34：信号产自哪条路，就只能证明哪条路〔本书〕

上面是测试路径，直接拼对象，信号是绿的；下面是生产路径，走真实加载与解析，结果是崩的。测试替身本来是为了跑得快。它替掉的，恰恰是装配那一段。而装配正是插件系统最容易出错的地方。速度换来一段没人看管的空白。

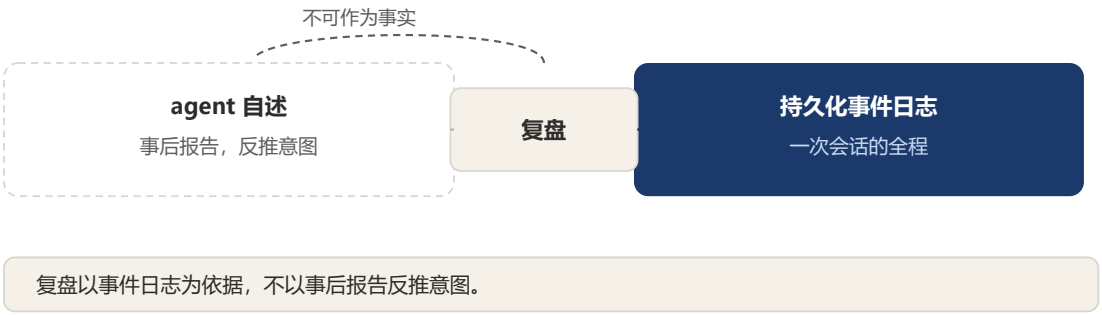


图 35：复盘的事实底座是日志，不是自述〔本书〕

左边是事后自述，画成虚线；右边是带序号的事件日志。复盘只从右边取事实。日志当时写下的顺序，就是复盘时的顺序。这是前面「单一事实源」那节结论的一次兑现。

15.3 两种说法的对照

小白说法	专业术语
姓名牌	Host 头校验
最高通行证	提权到最高权限
替业主签字	绕过人工审批
彩排走的后门	测试替身路径
门面亮灯就算开业	传输层就绪误当应用就绪
凭一个记号定罪	单信号归因
本子	事件日志

15.4 最容易理解错的地方

第一处：以为绿灯亮着就没事。四起事故全是漏报，没有一起误报。信号的意义由它产生的路径决定。

第二处：以为复盘是找出那行错代码。复盘问的是流程为什么放过了它。修一行是治标，流程漏的那一格才是本。

第三处：以为沙箱逃逸说明设计撒了谎。恰好相反，设计早把「不保证」写在了明处。漏洞不是打脸，是注脚。

第四处：以为一条证据就能归因。共享前缀不是协议。拿一个便宜的信号去顶一个贵的判断，是这类坑的共同形状。

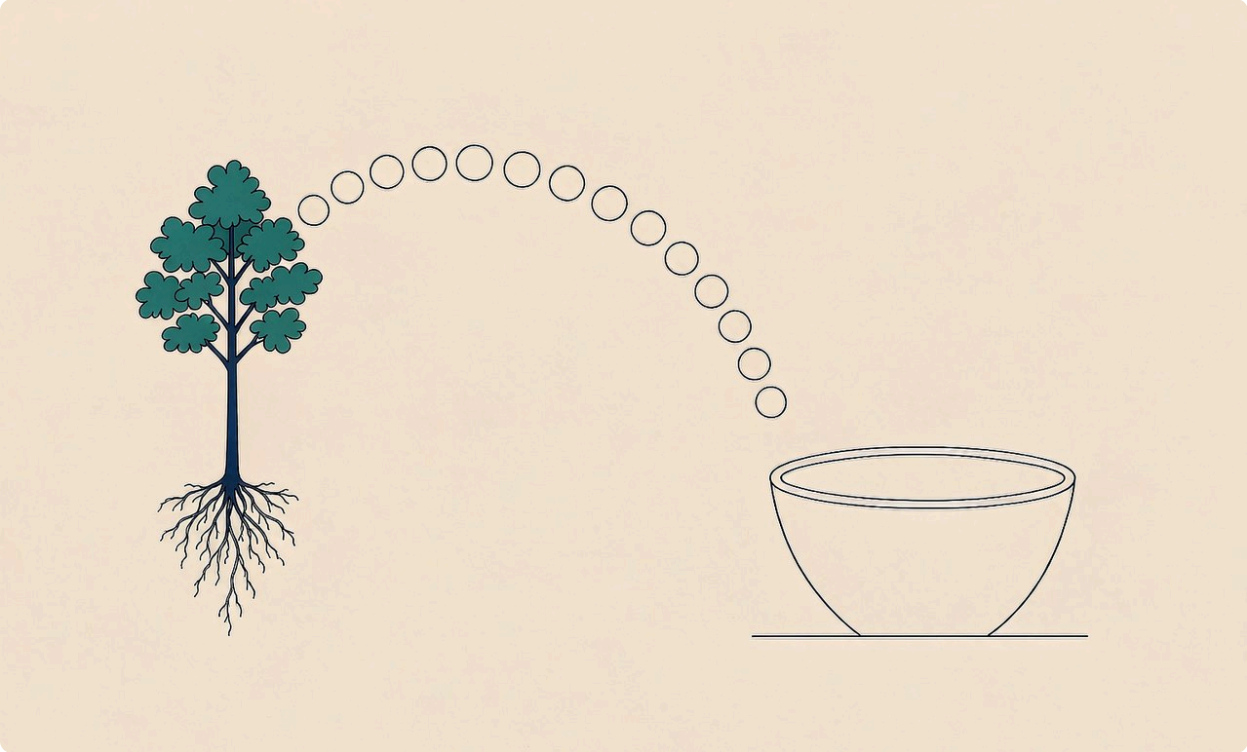
15.5 这条能搬去哪

四个自查动作。第一，挑一条最绿的测试。问它挂依赖的方式，和生产是不是同一条。第二，挑一次线上问题，看能不能只靠日志还原，不问当事人。

第三，挑一条归因规则，数一数它同时要求几项证据。第四，找出自己系统的「不保证」清单写在哪。没有，就补上。

四条里没有一条是让人多写测试。换的是问法：从信号够不够绿，换成它绿在哪条路上。

为什么这么设计？边界上的判断，全建立在信号之上。信号脱了上下文，边界就成了摆设。能搬去哪？任何靠自动化信号守门的流程：先对齐路径，再看数值。



连根拔起，种到另一块地里。

§16 不看代码，怎么知道它真的做到了

拆开机器数齿轮是一种验法。不拆，也能验。

总判据只有一句：删掉一个插件，系统回到装它之前的样子。它好在当场能验：挑一个，删掉，看有没有残留。五条判据各查一个侧面，能力、依赖、撤销、真源、信号，一样一条，互不重叠。用法也有讲究：别一次改五处，一条一条过。每过一条，可替换的程度就高一层。

16.1 小白版：五个当场能做的试验

一支装修队靠不靠谱，不用翻它的账本，做五个试验就知道。

试验一，换掉一个帮手，看要通知几个人。只改排班表上的一行，靠谱；挨个打电话通知，不靠谱。

试验二，把排班表上的顺序擦掉，看早上还能不能自己站好队。每人只写过一句「我需要谁」，队形就能自己推出来。擦掉就乱，说明顺序是背下来的。背的东西最怕加人。

试验三，请走一个人，看他交没交干净。钥匙、对讲机、门禁卡全交回，工位恢复原样，才算走完。交不干净，新人来了，同一件事两个人管，一件事干两遍。

试验四，把誊出来的纸全撕了，看能不能从底稿重新誊一份。能，说明底稿只有一本。不能，说明有两本底稿，两本迟早对不上。

试验五，别信奖状，先看奖状是哪场比赛发的。彩排发的奖状，证明不了正式上场。



图 36：五个试验各查一处，没有两个查同一件事〔本书〕

五个排成一列，各管一处：换不换得动、排不排得对、退不退得净、恢不恢得回、信不信得过。竖着读一遍，查的确实是五样不同的东西。

16.2 专业版：五条判据与一条链

判据一：换掉后端，消费方动不动。动作：挑一项能力换掉提供方，数要改几处。合格：消费方一行不动，它写下的是能力的名字，不是提供方的实现。还有一条硬信号：读一个没声明过的名字，应当当场抛错。这条排第一不是随意的，耦合朝着实现走，后面四条都无从谈起。

判据二：删掉手写顺序，还能不能排对。动作：删掉启动脚本里那串手写调用。合格：插件只声明「需要谁」，系统推出提供方在前、消费方在后，配置里两行对调，输出不变。不合格：一张手工排班表，加一个依赖得回去改表。还有一发暗雷：声明的服务缺席时系统不报错，插件停在等待态，什么也不做。

判据三：每个注册追不追得到一个撤销。动作：挑一个注册往上追。合格：注册挂在当前作用域上，撤销和注册共用一份代码，定时器、连接、监听器也包进去。不合格：卸不干净，旧监听器还挂在总线上，新实例又注册一份，同一件事被处理两次。热重载是这条判据的压力测试，改十次文件，进程里不该多留十份。

判据四：删掉派生数据，能不能重建。动作：把派生出来的状态全删了，看能不能从事件重新算回来。合格：真源只有一份，日志是事实，消息历史是投影，删了随时能再算。不合格：对话历

史存一份，事件日志存一份，两份真源迟早对不上。只有一份真源，卸载才有参照，才说得清回到装它之前是什么样。

判据五：这个信号产自哪条路。动作：挑一个最绿的信号，问它在哪条路径上产生，那条路和生产是不是同一条。前四条查机制，这一条查成绩单本身。它权重最高：管的是前四条的可信度。



图 37：前一条不过，后一条落不了地〔本书〕

决策树从顶上开始，不过就侧向停在一个阶段框里。能力不按名字分发，依赖就无处声明；注册撤销不了，真源再干净也回不到原点。五条不是一张清单，是一条链。

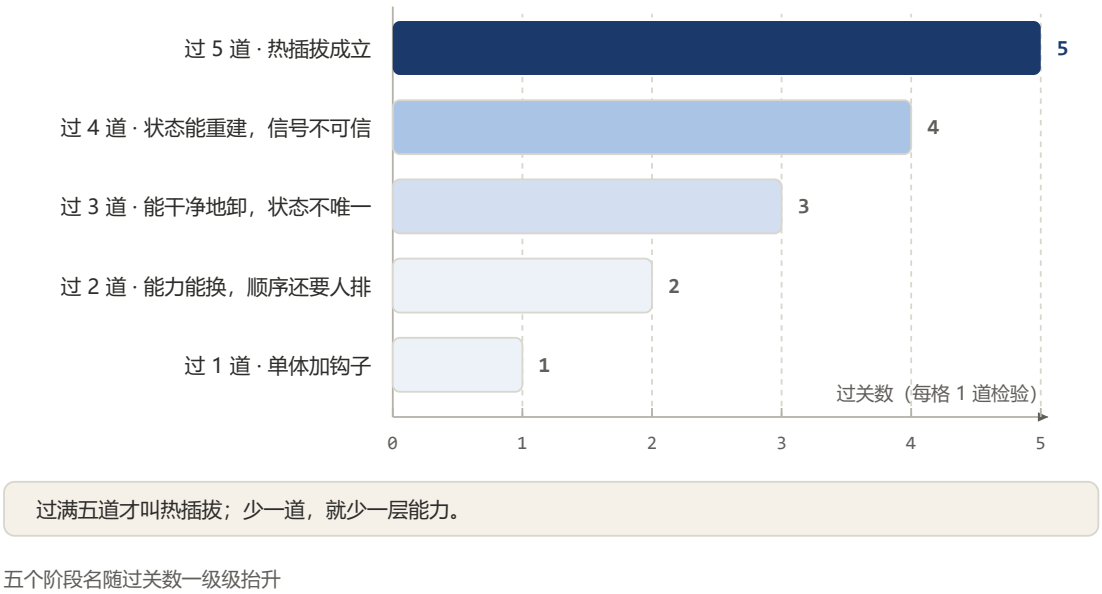


图 38：过几条就站在第几级，少一条少一层能力〔本书〕

横轴是过关数，纵轴是阶段名。过四条只是状态能重建，过满五条热插拔才成立。五条都过也不代表系统好，只代表它在这一件事上没有缺口。

16.3 两种说法的对照

小白说法	专业术语
把顺序擦掉	删除手写启动顺序
我需要谁	依赖声明
交还钥匙	撤销（dispose）
底稿	事件日志（单一事实源）
誊出来的纸	派生状态、投影
奖状	健康信号、指标

16.4 最容易理解错的地方

第一处：把五条当并列清单。它们是链，前一条是后一条的地基。顺序乱了，后面的判据落不了地。

第二处：以为过满五条系统就好。只代表可替换这件事没有缺口，别的什么也没说。

第三处：以为判据要读代码才能用。五条全是运行时的对照实验，删一个试试就知道。

第四处：以为第五条最轻。它排在末位是因为管得宽：前四条的成绩单可不可信，全归它查。

16.5 这条能搬去哪

不挑规模。一个三人维护的内部工具，也能一条一条对着查。查完先不用动代码，先知道自己在第几级。

为什么这么设计？五条合起来只回答一个问题：这套系统能不能在不重启的前提下，把自己改回原样。能搬去哪？任何想谈热替换、热重载的系统，先过这五条再开口。

§17 它在赌什么，代价是什么

全书到这里收梢。剩两件事。一是这套做法不是孤例，它和一套已有的理论讲的是同一件事。二是把它押的赌注和代价摆到明处。

17.1 小白版：隔壁行当的一本老账簿

这套组队的法子，不是这家公司发明的。隔壁行当有一本老账簿，记了很多年，翻开来，说的是同一回事。账簿从两个方向打量一支队伍。

横着看：谁需要谁。报名的时候只写名字不写人。喊的是「我要一个会拧螺丝的」，不是「我要小张」。人换了，名字还在，活照干。

竖着看：能不能请走，能不能接着干。请来的人，哪天请走，钥匙门禁全交回，工位恢复原样。活干到一半停下来，本子上记着呢，回来接着干，不用从头来。

这家队伍照着账簿押了两个宝。

一个押在师傅的脑子上：脑子会一茬一茬换，而且越换越快。所以脑子不能焊死在队伍里，得做成能换的。哪天脑子不换了，这个宝就押空了。

另一个押在搭伙的人身上：他们愿意多管一本排班表，愿意让中间隔着一层。哪天没人愿意了，这套组队法就没了主顾。

代价也摆在明处。新来的看这支队，看不懂它在干嘛，得先读懂排班表，把行为从排班表里重建出来。会这么组队的还不多，想找现成的搭伙人，选择很薄。

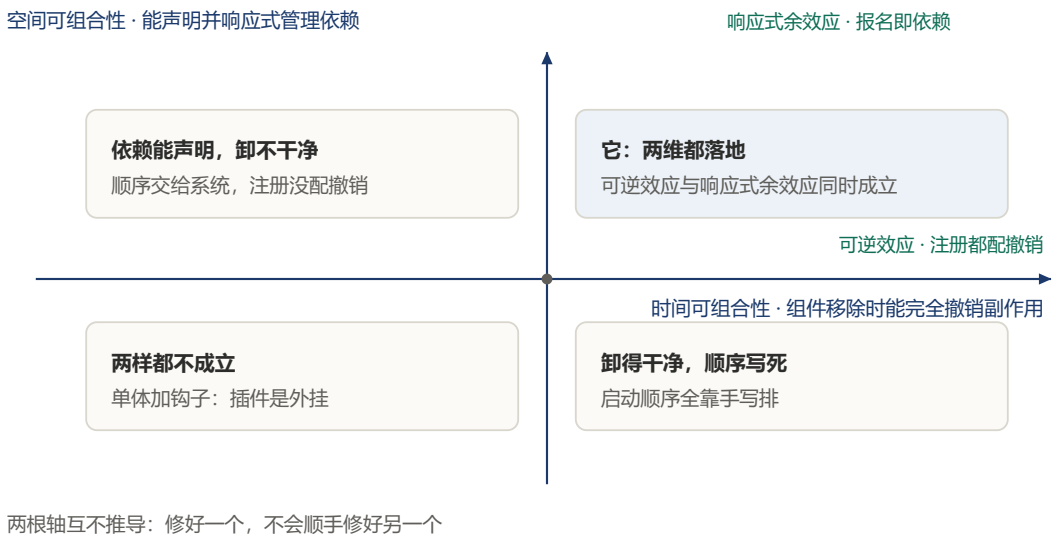


图 39：两根轴互相垂直，谁也推不出谁〔本书〕

横轴管谁需要谁，竖轴管请走与接着干。一支队伍可以请得干干净净，却把顺序写死在墙上；也可以顺序全自动，请走的人留下一串烂摊子。两根轴各管一摊，修好一根，不顺手修好另一根。

17.2 专业版：时空可组合与两个赌注

那套理论的正式名字叫时空可组合。一篇公开论文把它当成一个范式来讲，把这套工程当成范式的一个实现。

空间可组合性管谁需要谁：组件间的依赖能被声明，并被响应式地管理。机制名叫响应式余效应，落到工程上，就是按名字查的容器：消费方只写名字，装配方兑现。

时间可组合性管装与卸：组件被移除时，能完全撤销它带来的副作用。机制名叫可逆效应，每个上下文变换带一个逆操作。落到工程上，就是每个注册配一个撤销，以及会话能打断、能从中断处恢复。

两根轴垂直：一个系统可以卸得干干净净，却把顺序写死在脚本里；反过来也成立。时间和空间在这里是比喻，不是性能话题。



图 40：左列是能跑的工程，右列是形式定义，说的是同一件事〔本书〕

五行对照，每行一条双向箭头。按名查找、声明依赖、注册配撤销、单一真源、测试对齐生产路径，前面各节讲的每一条机制，都能在右列找到对应的一格。这就是同一件事的意思：不是比喻，是两种写法。

赌注一，模型会长期快速换代。这是模型插件化、外壳独立的全部理由。必答题各家都做：循环、上下文、工具接入、会话持久、权限沙箱。选答题只有它这么选：内核空置、模型插件化、日志可重放。换代一停，间接层的价值就跟着缩水。

赌注二，集成方愿意接受间接层的复杂度。选答题不是更好的答案，是把复杂度从实现方转移给了使用者。成品路线的系统替用户做决定，插座路线的系统给位子。位子上插什么，得自己懂行。

代价跟着赌注来。读懂成本高：装配清单自己不干活，读一遍清单看不出系统在干什么，行为得从配置里重建。生态还薄：插座卖的是位子，位子上现成的插头还不多。



五步顺序由依赖推出，不能乱序；第二步与第三步可并行，但都建在第一步之上。
回扣 §08 / §06 / §10 / §11 / §15

图 41：迁移的顺序不是自己排的，是从依赖里推出来的〔本书〕

每个方框是一个动作，箭头是谁得先做。起点是服务容器，它带出两条可并行的分支：声明顺序、注册配撤销。再往下是真源收成一份，末了是测试对齐生产路径。测试排在末尾不是最难，是它给前面所有步骤兜底。

17.3 两种说法的对照

小白说法	专业术语
老账簿	时空可组合理论
横着看，谁需要谁	空间可组合性（依赖声明与响应式管理）
竖着看，请走与接着干	时间可组合性（可逆效应、可恢复）
押宝	设计赌注
插座	可替换的集成点

17.4 最容易理解错的地方

第一处：把理论当成工程的背书。两者说的是同一件事，谁也不证明谁。工程能跑，理论能推，各站各的。

第二处：以为修好一根轴就够了。两根轴垂直，互不推导。只修一根，得到的恰好是一半。

第三处：以为选答题是更优解。它不是更好，是把复杂度换了个地方放。愿意接盘的觉得是灵活，不愿意的觉得是麻烦。

第四处：以为赌注稳赢。模型停止换代，或集成方不接受间接层，这套切法的理由就塌了。赌注之所以是赌注，因为它可能输。

17.5 这条能搬去哪

这套切法不挑语言，也不挑规模。两个动作就能起步。第一，列出系统里这几年真换过的东西，看换掉它们各要动几处；动得多的，就是下一个该做成能换的。第二，列出从来没换过的，问它们凭什么留在中心；答不上来的，请出去。

为什么这么设计？因为会换的东西焊死了，每一次换代都是一次伤筋动骨。能搬去哪？这套切法能搬去任何一个要长期演化的系统：先问什么会换，把它做成能换的；再问什么不换，留下当底子。

附录

附录 A 术语表

正文讲过的名词都收在这里。分三张表：第一张是全书类比的固定对应，第二章起不再重复解释；后两张按用途分组收术语，一句话说它是什么，标它在哪一节讲过。

一、师傅队伍：全书类比对照

这本书用一支修灯师傅的队伍讲整套系统。对应关系是固定的，全书通用。

队伍里的人	系统里的东西	讲过
师傅的脑子	模型（model），只会想，不会动手	§01
一整套人马	外壳（harness）	§01
拧螺丝的	工具（tool：读文件、写文件、跑命令）	§01
记笔记的	会话记录（session log）	§07
把门的	权限与沙箱（permission、sandbox）	§07
领班的	agent 循环（agent loop）	§07
临时工	插件（plugin）	§07
排班表	装配清单（assembly manifest）	§07
公司最里面那一层	内核（kernel）	§07
直接点名找小张	消费方直连实现	§07
换掉他要动几个人	改动半径	§07
公告栏	事件总线（谁都能贴，谁都能看）	§09
公司规矩、部门规矩、工位纸条	配置三层覆盖，小的压大的	§06
四套编制	四种形态（标准、PTC、极简、创造）	§06
备了替补的位子	接缝（有第二个实现的能力位）	§06
总账与抄件	日志是唯一事实，其余都是派生	§11
门禁卡入职发、离职收	注册即副作用，卸载即撤销	§10



图 14：领班和帮手站在同一层，一张排班表决定今天谁上〔重印自 §07〕

二、构件与角色

术语	一句话解释	讲过
外壳（harness）	模型之外、让它能干活的一整套	§02
框架（framework）	材料店：给材料，怎么盖随你	§02
脚手架（scaffolding）	图纸铺：给空架子，盖完就拆	§02
插件内核	内核只留原语，其余皆是插件	§07
服务键	一项能力占一个稳定名字	§08
事件总线	横向通信的唯一通道	§09
profile（形态）	一份具名组装，本质是一份名单	§06
bundle（组合包）	可复用的配置分发单元	§06
patch（修改条）	按名字定位一行，整块替换	§06
宿主	四种入口同进一个门的那一层	§05
子代理	领班叫来的外援小队	§12

三、机制与判断

术语	一句话解释	讲过
核心不变式	模型提议，外壳授权、执行、记录、恢复	§02
必答题	循环、上下文、工具接入、会话持久、权限沙箱	§03
选答题	内核空置、模型插件化、日志可重放	§03
改动半径	换掉一个组件要动几个地方	§07
幽灵状态	拆一半留一半，旧登记还挂着	§10
仅追加日志	只往后写、不改旧行的总账	§11
fork / resume	笔记复印一份，从那一页接着写	§11
三道闸	收紧、审批、隔离	§14
故障安全	出岔子时默认落在更严的那一档	§14
五条判据	不看代码判断一套系统做到没有	§16



图 4：必做活三家都做，拉开差距的是选做活〔重印自 §03〕

附录 B 全书图索引

全书 42 张图，按流水号收在这里。每张图标出它在哪一节、讲的是哪句结论。重印的图（同一句话在别处又用了一次）单列一张表。

序章

图号	结论	节
图 1	模型还是同一个模型，能不能干活全看外面那一圈	§00

第 1 部 · 进门

图号	结论	节
图 2	外壳上接模型、下接真实世界，模型的每句话都要经过它才能落地	§01

第 1 部 · 进门

图号	结论	节
图 3	三家都自称帮人造房，脾气完全不同	§02

第 1 部 · 进门

图号	结论	节
图 4	必做活三家都做，拉开差距的是选做活	§03

第 1 部 · 进门

图号	结论	节
图 5	四道难处，道道绕不过去	§04

图 6	**目标不是让左边不变，是左边变的时候右边不动**	**§04**

第 2 部 · 先看一眼

图号	结论	节
图 7	六层楼，每层只回答一个问题	§05

图 8	**四种来法都从门房进，门房不拿主意**	**§05**

图 9	**谁是真内核看依赖方向，不看代码量**	**§05**

第 2 部 · 先看一眼

图号	结论	节
图 10	规矩三层叠着来，小的压大的	§06

图 11	**四套编制共用一块薄底板，差别只在多雇了谁**	**§06**

图 12	**备了替补的位子，不等于替补已到岗**	**§06**

第 3 部 · 拆内核

图号	结论	节
图 13	师傅只会想，这七件事得队伍里另外有人补上	§07

图 14	**左边领班焊在公司里，右边他和帮手站平了**	**§07**

图 15	**换掉它，要动几个地方**	**§07**

第 3 部 · 拆内核

图号	结论	节
图 16	两边只认岗位名，谁也不认识谁	§08
图 17	**换实现只动排班表，用活的一行不改**	**§08**

第 3 部 · 拆内核

图号	结论	节
图 18	谁归谁管是一张图，谁跟谁说事是另一张	§09
图 19	**连开工次序都是推出来的，不靠私下协调**	**§09**

第 3 部 · 拆内核

图号	结论	节
图 20	发和收不是两份工，是同一张纸的正反两面	§10
图 21	**漏收一次，旧人还站在原处应答**	**§10**

第 3 部 · 拆内核

图号	结论	节
图 22	总账站着，对话记录只是它算出的一张抄件	§11
图 23	**两层草稿压住现抄的开销，扔了重抄都对**	**§11**

第 4 部 • 走一遍

图号	结论	节
图 24	要还原现场的落进笔记，过路的只喊一嗓子	§12
图 25	**七双手在场，指挥只有一个**	**§12**
图 26	**三个岔路口都卡在一步里，一趟的边界一个不碰**	**§12**

第 4 部 • 走一遍

图号	结论	节
图 27	五道手各有各的否决权，越靠后越只能决定看到什么	§13
图 28	**能喊停的手有好几只，递家伙的那只不在其中**	**§13**
图 29	**三种叫停，出口是同一张回话**	**§13**

第 5 部 • 碰边界

图号	结论	节
图 30	三道闸各挡一块，漏掉的地方互不补位	§14
图 31	**四个结果里只有一个放行，没人应答也算拒绝**	**§14**
图 32	**名字叫沙箱，管住的只有文件效果这一摊**	**§14**

第 5 部 • 碰边界

图号	结论	节
图 33	四张喜报全是绿的，对应的真实状态全错	§15
图 34	**信号产自哪条路，就只能证明哪条路**	**§15**
图 35	**复盘的事实底座是日志，不是自述**	**§15**

第 6 部 • 收

图号	结论	节
图 36	五个试验各查一处，没有两个查同一件事	§16
图 37	**前一条不过，后一条落不了地**	**§16**
图 38	**过几条就站在第几级，少一条少一层能力**	**§16**

第 6 部 • 收

图号	结论	节
图 39	两根轴互相垂直，谁也推不出谁	§17
图 40	**左列是能跑的工程，右列是形式定义，说的是同一件事**	**§17**
图 41	**迁移的顺序不是自己排的，是从依赖里推出来的**	**§17**

附录 C

图号	结论	节
图 42	角色看实现列，不看重要性	附录 C

重印的图

图号	结论	首用	重印于
图 1	师傅还是同一个师傅，配上人马才能干活	§00（图 1）	§01
图 2	人马不改师傅，只把师傅的力气引到活上	§01（图 2）	§02
图 14	领班和帮手站在同一层，一张排班表决定今天谁上	§07（图 14）	附录 A
图 4	必做活三家都做，拉开差距的是选做活	§03（图 4）	附录 A
图 12	有替换位，不等于备好了第二个实现	§06（图 12）	附录 C

附录 C 概念与代码位置对照

正文一律用角色名：模型适配器、工具注册表、会话日志。落到设计时，名字够用；想对着源码看时，名字就不够用了。

这一节补上那一层。前面是概念，后面是它在代码里叫什么、系统里挂的名字是什么。说白了，这是正文的下水道，删掉的代码细节都收在这里。

包名、文件路径、键名只在这里出现，别处一律换成角色名。表里的路径都写成仓库里的相对路径，从仓库根开始。

一、概念、代码与键

主表按四类排：内核四件套、能力类主要分组、装配类、边界类。

内核四件套

内核是删掉就没法干活的那几件。判断时数依赖，不数代码。

概念（角色名）	代码里叫什么	系统里挂的名字
会话日志	包 `core/session`	`ctx.sessions`
模型适配器	包 `llm/llm`	`ctx.llm`
Agent 接口与活跃注册表	包 `core/agent`	`ctx.agents`
工具注册表与执行流水线	包 `core/tools`	`ctx.tools`
提示词组装	包 `core/system-prompt`	`ctx.systemPrompt`
循环驱动器	包 `core/agent-loop`	`ctx.agentLoop`
作用域原语	包 `core/scope`	库，无 ctx 键

循环驱动器那一行值得多看一眼。它的键是 `ctx.agentLoop`，角色却标成组合包。这就是「循环也只是名单上一行」的落点。

能力类主要分组

能力层像一排插孔，一格一种能力。装哪个包，agent 就多哪只手。

概念（角色名）	代码里叫什么	系统里挂的名字
跑命令	分组 `shell/`，工具包 `shell/tool-bash`、`shell/tool-pwsh`	`ctx.shell`
读写文件	分组 `fs/`，工具包 `fs/tool-fs`	`ctx.fs`
接外部工具	分组 `mcp/`，包 `mcp/mcp-client`、`mcp/mcp-resources`	`ctx.mcpResources`
加载技能	分组 `skill/`，工具包 `skill/tool-skill`	`ctx.skills`
派小弟	分组 `subagent/`，工具包 `subagent/tool-subagent`	`ctx.subagents`
联网搜索与抓取	分组 `web/`，工具包 `web/tool-web`	`ctx.web`
压缩上下文	分组 `compaction/`，后端 `compaction/compaction-basic`	`ctx.compaction`
后台任务	分组 `jobs/`，后端 `jobs/jobs-local`	`ctx.jobs`
终端	分组 `terminal/`，工具包 `terminal/tool-terminal`	`ctx.terminals`

分组名和键名不一定同名。接外部工具挂的是 `ctx.mcpResources`，不是 `ctx.mcp`。

装配类

装配层只写名单。它的产物不是服务，是一棵树和一份清单。

概念（角色名）	代码里叫什么	系统里挂的名字
组装说明书	包 `package.json` 的 `dsh.profile` 字段，列出组合包	形态名 `web` / `headless` / `sdk` / `sdk-minimal` / `acp`
零件包	分组 `bundle/`，patch 文件由 `dsh.bundle` 字段指向	如 `bundle/base`
共享底座	包 `bundle/base`，清单在 `packages/bundle/base/cordis.patch.yml`	94 条 `- id:` 条目
形态层	包 `bundle/web-app`、`bundle/headless`、`bundle/sdk-app`、`bundle/acp-app`	各形态自己的 profile
不套底座的反例	包 `bundle/sdk-minimal`	独立 profile
启动器与热重载	分组 `boot/`，如 `boot/app-boot`、`boot/hmr`	——

边界类

边界层管哪些事不能做。它的活是刹车，产物是一道道否决口。

概念（角色名）	代码里叫什么	系统里挂的名字
收紧（策略把关）	分组 `guard/`；单调守卫在 `core/tools` 内	无独立键，排在 `tools/pre-execute` 之后
审批	包 `interaction/user-approval`	`ctx.approval`
隔离（沙箱）	分组 `sandbox/`，本机后端 `sandbox/sandbox-local`	`ctx.sandbox`
沙箱策略	包 `sandbox/sandbox-policy`	`ctx.sandboxPolicy`
权限预设	包 `interaction/permission-presets`	`ctx.permissionPresets`
凭据	分组 `credentials/`，本机后端 `credentials/credentials-local`	`ctx.credentials`
钩子桥接	分组 `hooks/`，如 `hooks/hooks-claude-code`	事件 `hook/invoked`、`hook/result`

附：事件名对照

正文讲模式，不讲事件名。想顺着事件找源码时，看这张。

概念	事件名	模式
一轮开始与结束	`turn/start` 、 `turn/end`	持久会话事件
一步开始与结束	`step/start` 、 `step/end`	持久会话事件
步骤前的决定	`agent/pre-step`	waterfall
请求发起前	`agent/request`	waterfall
模型流式输出	`llm/stream`	waterfall
工具调用前	`tools/pre-execute`	waterfall
工具环绕分派	`tools/execute`	waterfall
工具结果后	`tools/post-execute`	waterfall
工具权威结果	`tools/result`	emit
审批询问	`approval/request`	waterfall
会话事件广播	`session/event`	emit

二、92 个 ctx 键的角色分布

四种角色由官方标注，自己猜不出来。判断一个服务属于哪一类，看的就是角色这一列。

角色	官方叫法	键数	有没有替换位
core	核心主干服务	55	没有，单实现
seam	可替换的能力 seam	33	有
service	独立服务	3	没有
bundle	组合包	1	没有
合计	——	92	——



图 42：角色看实现列，不看重要性〔本书〕

图上四行，右列是实现列。核心主干、独立服务、组合包三格是空的。只有 seam 那格不空。

59 个键没有替换位，33 个有。能换的不到四成。

能换的那 33 个，还得再数一遍。

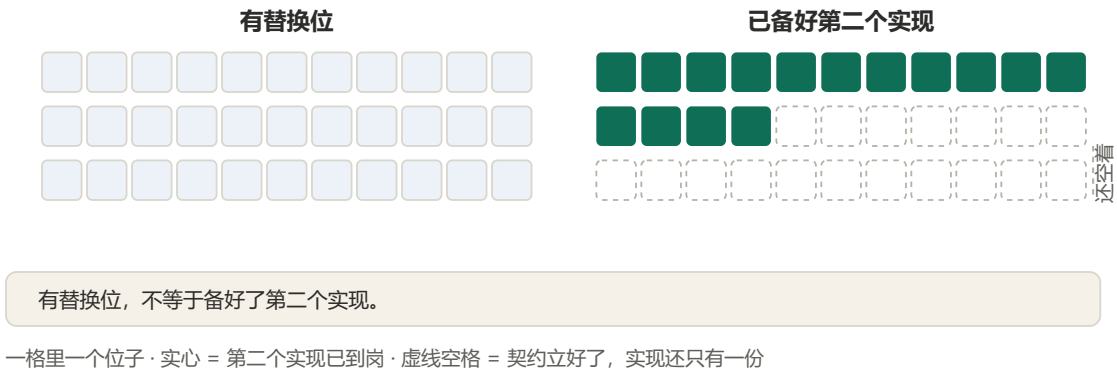


图 12：有替换位，不等于备好了第二个实现〔重印自 §06〕

横轴 33 格，每格一个 seam。绿色的 15 格已有两个以上平级实现，灰色的 18 格没有。

这 18 格还要再分。15 个只有一份实现，另外 3 个连一份都挂在拥有者身上。那 3 个是 `ctx.authorization`、`ctx.userQuestions`、`ctx.approval`。

留了替换位和备好第二个实现，是两笔账。前者在立契约时就付掉，后者可以晚付。

两笔账，分开算。

三、想核这些数字时，去哪儿看

口径集中在这里。想自己复现，按下面几条走。

- 版本：所有统计以 `dsh-0.2.0-rc.2`（2026-09-29，`master`）为准。上游在预览期，换了版本数字会动。
- 包数：读工作区声明 `pnpm-workspace.yaml` 里的 `packages/*/*` 一行，逐个遍历 `packages/<分组>/<包>/package.json`，按包名去重。
- 代码行：数 `.ts` / `.tsx` / `.js` / `.jsx` / `.py`，含测试文件，排除 `node_modules/` 与 `dist/`。
- 依赖边：读每个 `package.json` 的 `dependencies` / `peerDependencies` / `devDependencies`，只看 `@deepseek-ai/dsh-` 前缀的包间引用。
- ctx 键与角色：读仓库里那份能力表，文件名 `capability-seams.zh.md`。按行数 `ctx.<key>` 与角色列；实现列里平级实现包有两个以上的，算备好了第二个实现。
- 装配清单条数：数 `packages/bundle/base/cordis.patch.yml` 里以 `- id:` 开头的行。
- 那 94 行代码：`packages/bundle/base/src/index.ts` 9 行，加 `packages/bundle/base/tests/base.spec.ts` 85 行。

换一种数法，同一个对象会得到另一个数。上面每条都写死了包含与排除，照着重跑能对上。